Universidad Mariano Gálvez de Guatemala

Centro Universitario de Puerto Barrios

Facultad de Ingeniería en Sistemas

Redes de Computadoras I

Ing. Miguel Angel Cardona Perez

Sistema Virtualizado con GNS3

Proyecto

Integrantes:

Hennrry Geovanny Canahuí Gomez 0909-23-258

Mike Jason René Martinez Barrientos 0909-22-15059

Edgar Alejandro Pérez Monroy 0909-23-631

Puerto Barrios, Izabal, 31 de octubre 2026

Estructura del Proyecto

Nivel 1: Capa Física y Dispositivos

En este apartado se detalla la infraestructura, la asignación de interfaces físicas/virtuales y la topología utilizada:

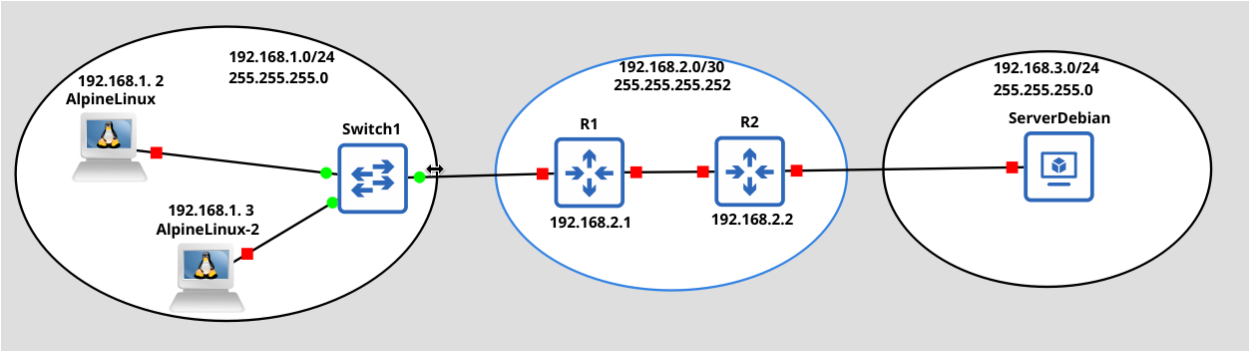


Tabla de Conexiones Físicas:

Dispositivo Origen	Interfaz Origen	Dispositivo Destino	Interfaz Destino	Red / Propósito
Alpine Linux	eth0	Switch1	Ethernet 0	LAN Clientes: 192.168.1.0/24 (255.255.255.0)
Alpine Linux-2	eth0	Switch1	Ethernet 1	LAN Clientes: 192.168.1.0/24 (255.255.255.0)
Switch1	Ethernet 2	R1 (Cisco 3725)	FastEthernet0/0	LAN Clientes: 192.168.1.0/24 (255.255.255.0)
R1 (Cisco 3725)	FastEthernet0/1	R2 (Cisco 3725)	FastEthernet0/0	Enlace WAN Punto a Punto: 192.168.2.0/30 (255.255.255.252)
R1 (Cisco 3725)	FastEthernet0/1	ServerDebian	Ethernet 1	LAN Servidores: 192.168.3.0/24 (255.255.255.0)

Nivel 2: Capa de Enlace de Datos (L2) y Direccionamiento Lógico (L3).

Tabla de Configuración de Interfaces y Direccionamiento IP:

Dispositivo	Interfaz	Dirección IP	Máscara de Subred	Gateway	Modo / Función
DebianLinux	eth0	192.168.1.2	255.255.255.0 (/24)	192.168.1.1	Estático / Cliente LAN 1
DebianLinux-2	eth0	192.168.1.3	255.255.255.0 (/24)	192.168.1.1	Estático / Cliente LAN 2
Switch 1	Eth0 –Eth2	N/A (L2)	N/A	N/A	Access (VLAN 1) Unmanaged Switch
R1	Fa0/0	192.168.1.1	255.255.255.0 (/24)	N/A	Gateway LAN Clientes
R1	Fa0/1	192.168.2.1	255.255.255.252 (/30)	N/A	Enlace Punto a Punto WAN
R2	Fa0/0	192.168.2.2	255.255.255.252 (/30)	N/A	Enlace Punto a Punto WAN
R2	Fa0/1	192.168.3.1	255.255.255.0 (/24)	N/A	Gateway LAN Servidores
Server Debian	ens1 (Eth1)	192.168.3.2	255.255.255.0 (/24)	192.168.3.1	Estático / Servidor de Servicios

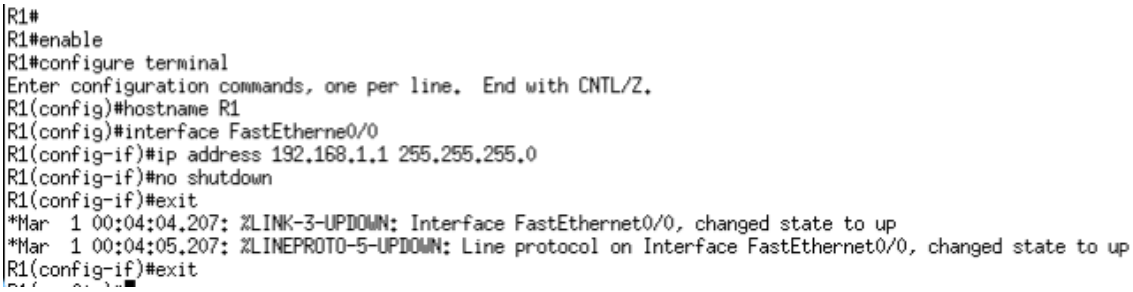
Tabla de Enrutamiento Estático:

Router	Red Destino	Máscara de Subred	Siguiente Salto (Next-Hop)	Interfaz de Salida	Propósito
R1	192.168.3.0	255.255.255.0 (/24)	192.168.2.2	FastEthernet0/1	Ruta hacia LAN Servidores
R2	192.168.1.0	255.255.255.0 (/24)	192.168.2.1	FastEthernet0/0	Ruta hacia LAN Clientes

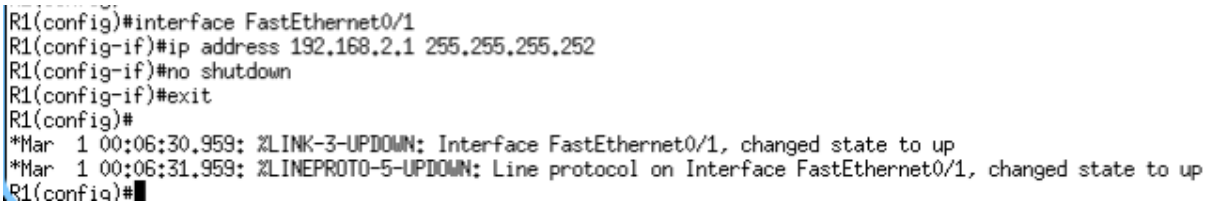
Configuracion de las maquinas:

Configuraciones dispositivo R1:

Interfaz hacia la LAN A (Conectada al Switch1)



Interfaz hacia el enlace Punto a Punto (Conectada a R2)



Ruta estática hacia la LAN B (Red de Debian detras de R2)

```
R1(config)#
R1(config)#ip route 192.168.3.0 255.255.255.0 192.168.2.2
R1(config)#end
R1#w
*Mar  1 00:08:47.167: %SYS-5-CONFIG_I: Configured from console by console
R1#write memory
Building configuration...
[OK]
R1#
R1#
```

verificación de configuraciones

```
R1#
R1#
R1#show ip interface brief
Interface                  IP-Address      OK? Method Status                Protocol
FastEthernet0/0            192.168.1.1     YES manual up                    up
FastEthernet0/1            192.168.2.1     YES manual up                    up
R1#
```

Configuraciones dispositivo R2:

Interfaz hacia el enlace Punto a Punto (Conectada a R1)

```
R2#
R2#
R2#enable
R2#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#interface FastEthernet0/0
R2(config-if)#ip address 192.168.2.2 255.255.255.252
R2(config-if)#no shutdown
R2(config-if)#
*Mar  1 00:03:58.607: %LINK-3-UPDOWN: Interface FastEthernet0/0, changed state to up
*Mar  1 00:03:59.607: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
R2(config-if)#exit
```

Interfaz hacia la LAN B (Conectada a ServerDebian)

```
R2(config)#
R2(config)#
R2(config)#
R2(config)#
R2(config)#
R2(config)#interface FastEthernet0/1
R2(config-if)#ip address 192.168.3.1 255.255.255.0
R2(config-if)#no shutdown
R2(config-if)#
*Mar  1 00:06:15.175: %LINK-3-UPDOWN: Interface FastEthernet0/1, changed state to up
*Mar  1 00:06:16.175: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up
R2(config-if)#exit
```

Ruta estática hacia la LAN A (Red de los Alpine detras de R1)

```
R2(config)#
R2(config)#
R2(config)#
R2(config)#ip route 192.168.1.0 255.255.255.0 192.168.2.1
R2(config)#end
R2#
*Mar  1 00:07:48.443: %SYS-5-CONFIG_I: Configured from console by console
R2#write memory
Building configuration...
[OK]
R2#
R2#
```

verificación de configuraciones

```
R2#
R2#
R2#show ip interface brief
Interface                  IP-Address      OK? Method Status                Protocol
FastEthernet0/0            192.168.2.2     YES manual up                    up
FastEthernet0/1            192.168.3.1     YES manual up                    up
```

configuraciones serverdebian:

Asignar IP y levantar la interfaz:

```
root@debian:~# ip addr add 192.168.3.2/24 dev ens1
root@debian:~# ip link set ens1 up
root@debian:~# ip route add default via 192.168.3.1 dev ens1
root@debian:~#
root@debian:~#
```

alpineLinux (cliente 1):

```
/ #
/ #
/ # ip addr add 192.168.1.2/24 dev eth0
/ # ip link set eth0 up
/ # ip route add default via 192.168.1.1
/ #
/ #
/ #
```

alpineLinux-2 (cliente 2):

```
/ #
/ #
/ # ip addr add 192.168.1.3/24 dev eth0
/ # ip link set eth0 up
/ # ip route add default via 192.168.1.1
/ #
/ #
/ #
```

Controlador de Dominio

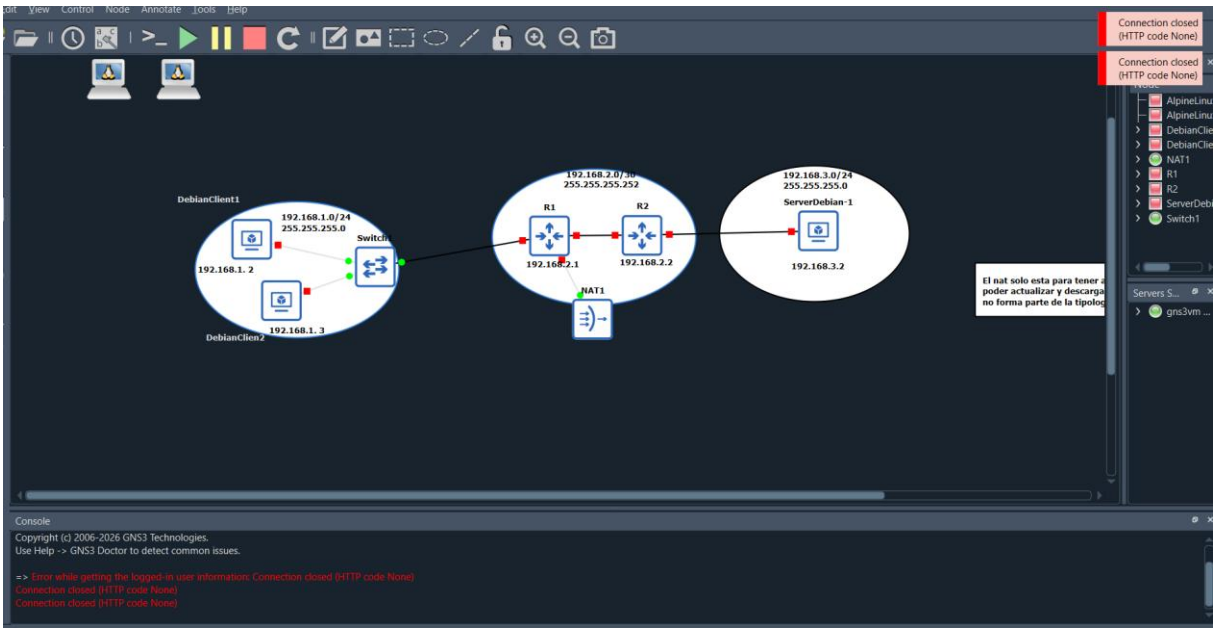
Registra la evolución posterior de la topología, la migración de los clientes Alpine a Debian, la implementación del controlador de dominio, las pruebas de autenticación y el avance actual del servicio DHCP.

Evolución de la topología

La topología base documentada inicialmente se mantuvo en su estructura de tres segmentos IPv4, pero los dos clientes Alpine Linux fueron sustituidos por máquinas Debian 13 con entorno gráfico XFCE. La red de clientes continúa siendo 192.168.1.0/24, el enlace punto a punto entre R1 y R2 utiliza 192.168.2.0/30 y la LAN de servidores utiliza 192.168.3.0/24. R1 conserva la salida centralizada a Internet mediante NAT1 y R2 mantiene la interconexión hacia el servidor Debian.

Figura 1

Topología actual del proyecto en GNS3.



Nota. Se observan DebianClient1 y DebianClient2 en la LAN 192.168.1.0/24, R1 y R2 unidos por 192.168.2.0/30, ServerDebian-1 en 192.168.3.0/24 y NAT1 conectado únicamente a R1.

Credenciales y accesos del laboratorio

Las siguientes credenciales corresponden exclusivamente al entorno de simulación del Proyecto Redes I. Se incluyen para facilitar la importación, configuración y comprobación del proyecto por parte de otra persona.

Sistema / Servicio	Usuario	Contraseña
Servidor-Ubuntu (configuración inicial)	root	12345
Servidor-Ubuntu (configuración inicial)	debian	12345
Clientes Debian – usuario root	root	DebianRoot2026A
Cliente Debian	clienteadmin	ClienteAdmin2026A
Dominio REDESI.TEST	usuario1	UsuarioRedes2026A
Dominio REDESI.TEST	usuario2	UsuarioRedes2026B
GNS3 VM	gns3	gns3
Samba Active Directory	Administrator	RedesIDC2026A
FTPS / vsftpd	ftpuser	FTPRedes2026A
GNS3 Controller HTTPS/API	admin	admin
dc1 – acceso SSH root	root	Contraseña root configurada localmente (no quedó registrada de forma inequívoca)
Correo / Postfix-Dovecot	correo1	CorreoRedes2026A
Correo / Postfix-Dovecot	correo2	CorreoRedes2026B

Conexiones SSH y accesos de administración

Para facilitar la administración después de importar el laboratorio, se documentan los accesos SSH utilizados desde Windows PowerShell. Los tres accesos atraviesan la GNS3 VM mediante ProxyJump y llegan a las publicaciones NAT de R1. La primera autenticación corresponde a gns3@192.168.56.102 y utiliza la contraseña gns3.

Equipo	Comando	Credenciales	Observación
GNS3 VM	ssh gns3@192.168.56.102	gns3 / gns3	Acceso directo a la VM de GNS3.
dc1 / ServerDebian	ssh -J gns3@192.168.56.102 -p 2223 root@192.168.122.100	1.º gns3 / gns3; 2.º root / contraseña local de dc1	R1 publica TCP/2223 → 192.168.3.2:22.
DebianClient1	ssh -J gns3@192.168.56.102 -p 2222 clienteadmin@192.168.122.100	1.º gns3 / gns3; 2.º clienteadmin / ClienteAdmin2026A	R1 publica TCP/2222 → 192.168.1.2:22.
DebianClient2	ssh -J gns3@192.168.56.102 -p 2224 clienteadmin@192.168.122.100	1.º gns3 / gns3; 2.º clienteadmin / ClienteAdmin2026A	R1 publica TCP/2224 → 192.168.1.3:22.
Elevar a root en clientes	su -	root / DebianRoot2026A	Se ejecuta después de entrar como clienteadmin.

Nota. El usuario admin/admin corresponde al controlador HTTPS/API de GNS3 y no sustituye a gns3/gns3 para el salto SSH. La contraseña de root actual de dc1 no se infiere ni se modifica en esta documentación porque no quedó consignada de forma inequívoca en las evidencias; debe usarse la contraseña establecida localmente en esa VM.

Datos del dominio

- Dominio: REDESI.TEST
- Controlador de dominio: dc1.redesi.test
- Dirección IP del controlador: 192.168.3.2
- Cliente 1: cliente1 – 192.168.1.2
- Cliente 2: cliente2 – 192.168.1.3

Nota: Estas credenciales son únicamente para el laboratorio académico y no deben utilizarse en sistemas reales.

Plan de direccionamiento actual

Dispositivo	Interfaz / función	Dirección IP	Gateway / siguiente salto
GNS3 VM	Administración	192.168.56.102	Red host-only

NAT1	Gateway NAT	192.168.122.1/24	Salida a Internet
R1	Fa0/0 – LAN clientes	192.168.1.1/24	—
R1	Fa0/1 – enlace R2	192.168.2.1/30	—
R1	Fa1/0 – NAT outside	192.168.122.100/24	192.168.122.1
R2	Fa0/0 – enlace R1	192.168.2.2/30	192.168.2.1
R2	Fa0/1 – LAN servidores	192.168.3.1/24	—
dc1.redesi.test	ens1 – servidor	192.168.3.2/24	192.168.3.1
cliente1.redesi.test	enp2s0 – cliente	192.168.1.2/24	192.168.1.1
cliente2.redesi.test	enp2s0 – cliente	192.168.1.3/24	192.168.1.1

Enrutamiento, NAT e Internet

Se completó el enrutamiento estático entre las redes internas. R1 mantiene la ruta hacia 192.168.3.0/24 mediante 192.168.2.2 y R2 mantiene la ruta hacia 192.168.1.0/24 mediante 192.168.2.1. Además, R1 funciona como único punto de salida a Internet mediante PAT, evitando la necesidad de agregar un segundo nodo NAT a la topología.

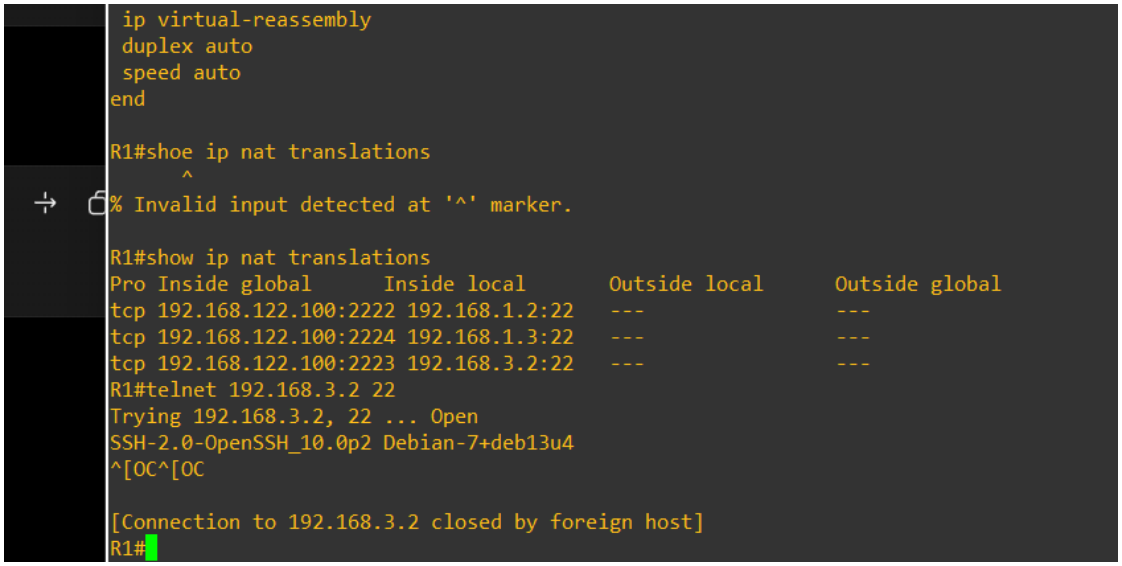
```
R1:
 ip route 192.168.3.0 255.255.255.0 192.168.2.2
 ip route 0.0.0.0 0.0.0.0 192.168.122.1

R2:
 ip route 192.168.1.0 255.255.255.0 192.168.2.1
 ip route 0.0.0.0 0.0.0.0 192.168.2.1
```

Las pruebas de conectividad confirmaron comunicación extremo a extremo entre los clientes y dc1, así como acceso a Internet desde los clientes y el servidor. También se publicaron puertos SSH en R1 con fines de administración remota, aunque durante la etapa actual se detectó una incidencia con el acceso desde la GNS3 VM hacia los puertos publicados. La comunicación interna R1 → R2 → dc1 por TCP/22 sí fue validada correctamente mediante Telnet.

Figura 2

Validación de conectividad SSH entre R1 y dc1.



Nota. R1 abre correctamente una conexión TCP al puerto 22 de 192.168.3.2 y recibe el banner de OpenSSH, confirmando que el servicio SSH del servidor es alcanzable desde la red interna.

Pruebas de verificación de enrutamiento, NAT e Internet

Al finalizar la configuración de enrutamiento y NAT se realizaron pruebas desde routers, servidor y clientes. El objetivo fue comprobar alcance entre las tres redes, salida a Internet y administración remota mediante las publicaciones NAT de R1.

Elemento comprobado	Prueba / comando	Resultado observado	Estado
Ruta R1 → LAN de servidores	R1: ping 192.168.3.2 / telnet 192.168.3.2 22	Alcance exitoso y banner OpenSSH del servidor dc1.	**APROBADO**
Cliente1 → dc1	ping -c 4 192.168.3.2	4 respuestas, 0 % de pérdida.	**APROBADO**
Cliente2 → dc1	ping -c 4 192.168.3.2	4 respuestas, 0 % de pérdida.	**APROBADO**
Salida a Internet	Cliente1/Cliente2: ping -c 4 8.8.8.8	Acceso externo conservado mediante R1 y NAT1.	**APROBADO**
PAT / publicaciones SSH	R1: show ip nat translations	Mapeos internos y publicaciones TCP 2222, 2223 y 2224 disponibles.	**APROBADO**
SSH Cliente1	ssh -J gns3@192.168.56.102 -p 2222 clienteadmin@192.168.122.100	Inicio de sesión correcto en cliente1.	**APROBADO**
SSH dc1	ssh -J gns3@192.168.56.102 -p 2223 root@192.168.122.100	Inicio de sesión correcto después de corregir la ruta de retorno.	**APROBADO**
SSH Cliente2	ssh -J gns3@192.168.56.102 -p 2224 clienteadmin@192.168.122.100	Inicio de sesión correcto en cliente2.	**APROBADO**

Servidor Debian como controlador de dominio

ServerDebian-1 fue configurado con hostname dc1 y FQDN dc1.redesi.test. La interfaz interna ens1 utiliza 192.168.3.2/24 y la puerta de enlace 192.168.3.1. La ruta de salida del servidor se mantiene a través de R2, R1 y NAT1, por lo que el servidor participa en la topología real del proyecto y no depende de una conexión NAT directa adicional.

Samba fue aprovisionado como controlador de dominio Active Directory para el dominio redesi.test y el realm Kerberos REDESI.TEST. Se verificó el servicio samba-ad-dc activo, la resolución del nombre dc1.redesi.test hacia 192.168.3.2 y la existencia de registros SRV de LDAP y Kerberos. El DNS de Samba utiliza un reenviador externo para resolver nombres de Internet.

Parámetro	Configuración actual
Hostname	dc1
FQDN	dc1.redesi.test
Dominio DNS	redesi.test
Realm Kerberos	REDESI.TEST
IP del DC	192.168.3.2/24
Gateway	192.168.3.1
DNS local del servidor	127.0.0.1
DNS forwarder de Samba	1.1.1.1

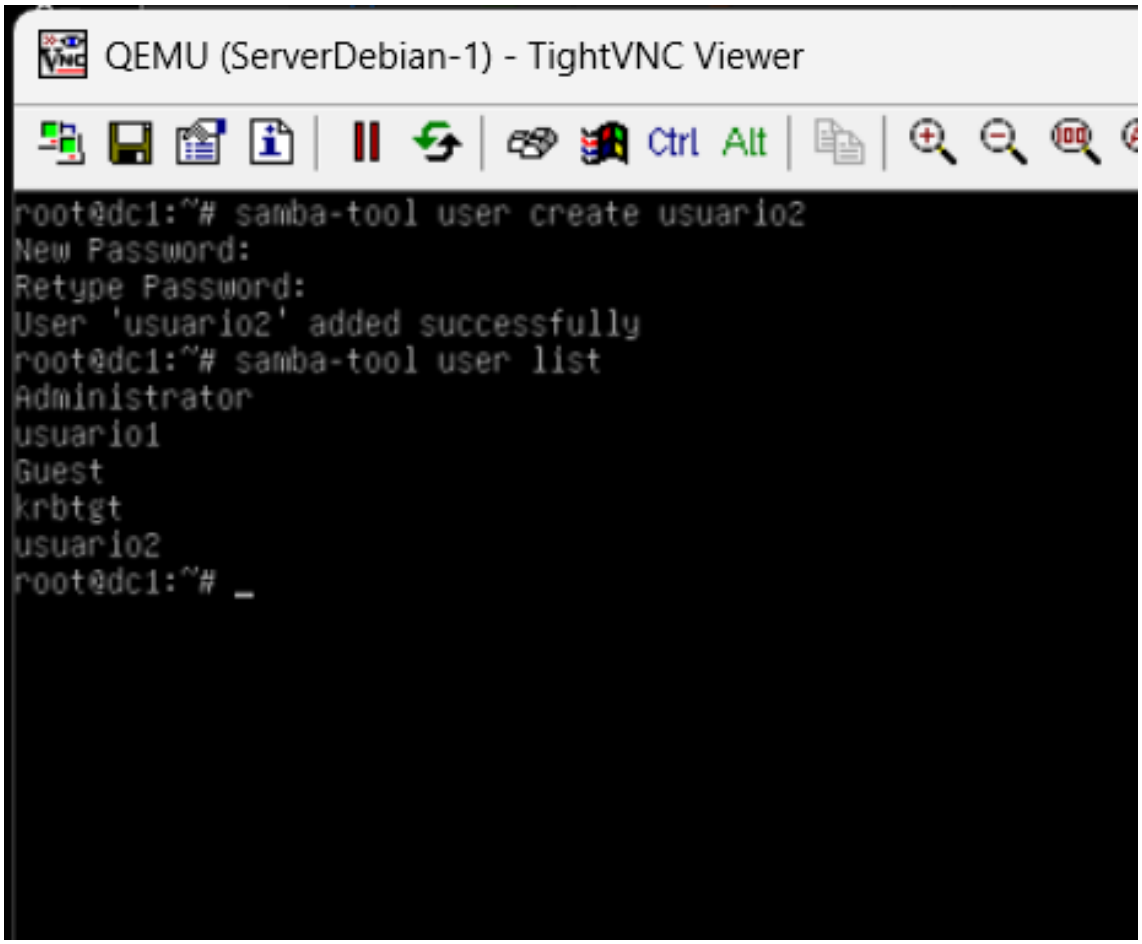
Usuarios de Active Directory

Además de la cuenta administrativa del dominio, se crearon cuentas de prueba para demostrar la administración centralizada de usuarios. usuario1 ya había sido utilizado para validar el acceso desde los clientes. Posteriormente se creó usuario2 para mostrar una segunda identidad de dominio y realizar una demostración más clara utilizando un usuario diferente en cliente2.

```
samba-tool user create usuario2
samba-tool user list
samba-tool user show usuario2
```

Figura 3

Creación y verificación de usuario2 en Samba Active Directory.



Nota. La salida confirma que usuario2 fue agregado correctamente y aparece dentro del contenedor Users del dominio redesi.test.

Integración de DebianClient1 y DebianClient2 al dominio

Los dos clientes actuales utilizan Debian 13 con XFCE y están unidos de forma independiente al dominio redesi.test. La integración se realizó con realmd, SSSD, Kerberos y PAM. Ambos equipos quedaron registrados como kerberos-member y poseen cuentas de computadora diferentes en Active Directory, por lo que la clonación de cliente2 no genera conflicto con CLIENTE1$.

```
realm discover redesi.test
realm join -v --install=/ -U Administrator redesi.test
```

```
realm list
systemctl restart sssd
sss_cache -E
pam-auth-update --enable mkhomedir
```

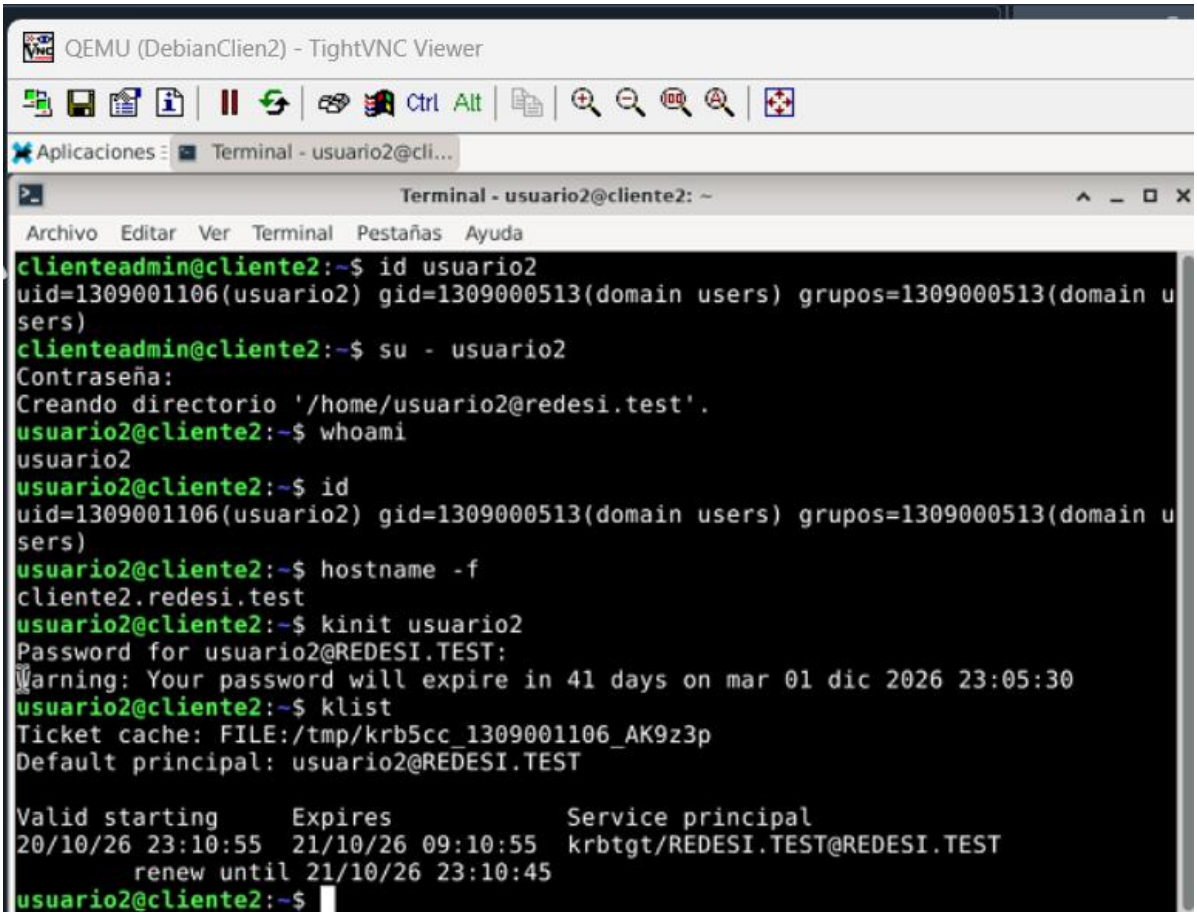
Se configuró use_fully_qualified_names = False en SSSD para permitir el uso de nombres cortos, por ejemplo usuario1 y usuario2, y pam_mkhomedir para crear el directorio personal durante el primer inicio de sesión. LightDM se configuró para permitir ingreso manual de credenciales del dominio.

Validación de usuario2 en cliente2

En cliente2 se confirmó que SSSD resuelve la cuenta usuario2, que la autenticación PAM funciona mediante su, que se crea automáticamente el directorio /home/usuario2@redesi.test y que el usuario obtiene un ticket Kerberos válido. La prueba whoami devolvió usuario2, id mostró la pertenencia a domain users y hostname -f confirmó cliente2.redesi.test.

Figura 4

Autenticación de usuario2 y ticket Kerberos en cliente2.



Nota. La evidencia muestra id usuario2, creación del directorio personal, whoami, hostname -f, kinit usuario2 y klist con Default principal usuario2@REDESI.TEST.

Pruebas de verificación del controlador de dominio, DNS y Kerberos

Una vez aprovisionado Samba AD y unidos ambos clientes al dominio, se ejecutaron las siguientes pruebas. Estas validaciones comprueban que el servidor funciona simultáneamente como controlador de dominio, DNS interno y KDC Kerberos, y que las identidades del dominio pueden utilizarse desde los clientes Debian.

Elemento comprobado	Prueba / comando	Resultado observado	Estado
Samba AD DC	systemctl status samba-ad-dc --no-pager	Servicio active (running).	**APROBADO**
Cuentas del dominio	samba-tool user list	Administrator, usuario1, usuario2 y cuentas internas del dominio visibles.	**APROBADO**
Equipos unidos al dominio	samba-tool computer list	CLIENTE1$ y CLIENTE2$ registrados de forma independiente.	**APROBADO**

Elemento comprobado	Prueba / comando	Resultado observado	Estado
DNS A del DC	host dc1.redesi.test	dc1.redesi.test → 192.168.3.2.	**APROBADO**
DNS SRV LDAP	host -t SRV _ldap._tcp.redesi.test	Servicio LDAP anunciado por dc1.redesi.test:389.	**APROBADO**
DNS SRV Kerberos	host -t SRV _kerberos._udp.redesi.test	Servicio Kerberos anunciado por dc1.redesi.test:88.	**APROBADO**
Cliente1 / usuario1	realm list; id usuario1; kinit usuario1; klist	kerberos-member, domain users y ticket usuario1@REDESI.TEST.	**APROBADO**
Cliente2 / usuario2	realm list; id usuario2; kinit usuario2; klist	kerberos-member, domain users y ticket usuario2@REDESI.TEST.	**APROBADO**
Inicio de sesión gráfico	LightDM + credenciales de dominio	Ingreso exitoso y creación automática del directorio personal con pam_mkhomedir.	**APROBADO**

Optimización de recursos de las máquinas virtuales

Antes de continuar con los servicios se revisó el consumo de recursos debido a la carga simultánea de R1, R2, dc1, DebianClient1 y DebianClient2. Durante las instalaciones gráficas se observaron episodios de soft lockup en la GNS3 VM, por lo que se ajustaron los recursos del laboratorio. La GNS3 VM quedó con aproximadamente 6024 MB de memoria y 4 vCPU, mientras que los clientes Debian se ampliaron a 1.5 GB de RAM para ejecutar XFCE, FileZilla y Thunderbird con mayor estabilidad.

Esta optimización no cambia el direccionamiento, las cuentas del dominio ni los discos qcow2. Su objetivo es permitir que los equipos principales permanezcan activos durante la demostración y que los clientes puedan ejecutar las herramientas gráficas sin alterar los servicios de red.

Incidencia de administración HTTPS en GNS3

El controlador remoto de GNS3 fue configurado mediante HTTPS en 192.168.56.102:443. Después del cambio, el cliente gráfico comenzó a mostrar de manera intermitente el mensaje Connection closed (HTTP code None) al iniciar o detener nodos. Las pruebas demostraron que el servidor sí ejecuta la operación y posteriormente emite el evento node.updated con el estado correcto del nodo; por tanto, la incidencia afecta la respuesta de la interfaz de administración y no la funcionalidad de la topología.

La consulta HTTPS /v3/version respondió HTTP 200 OK y la autenticación del controlador generó correctamente un token de acceso. Se decidió dejar esta incidencia documentada y continuar con el proyecto, ya que no afecta AD, DNS, Kerberos, enrutamiento ni los servicios que se están implementando.

Implementación de DHCP – avance de la entrega semanal

El siguiente servicio en implementación es DHCP. El diseño conserva dc1 como servidor central y utilizará R1 como DHCP Relay para la LAN 192.168.1.0/24. Con este enfoque, los clientes obtendrán automáticamente dirección IPv4, gateway, DNS y dominio sin instalar un segundo servidor en la red de clientes.

Opción DHCP	Valor planificado
Subred de clientes	192.168.1.0/24
Gateway entregado	192.168.1.1
Servidor DNS	192.168.3.2
Nombre de dominio	redesi.test
Pool dinámico	192.168.1.100 – 192.168.1.150
Reserva cliente1	192.168.1.2
Reserva cliente2	192.168.1.3
Servidor DHCP	dc1.redesi.test / 192.168.3.2
DHCP Relay	R1 Fa0/0 → 192.168.3.2

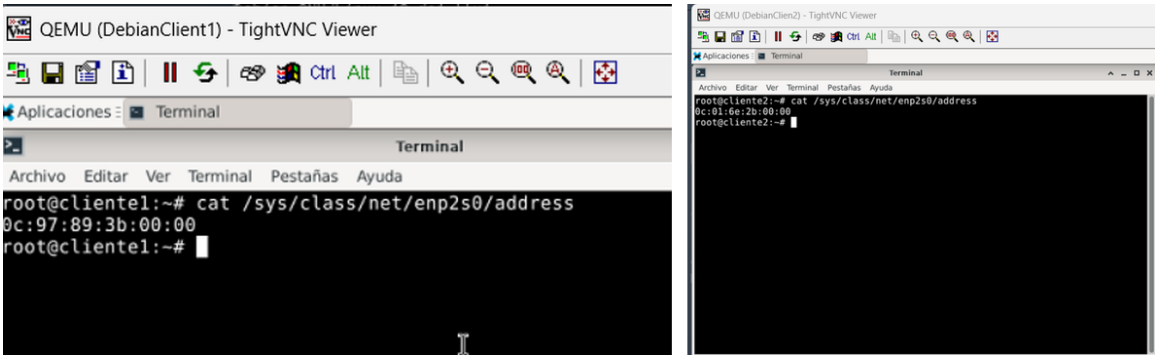
Identificación de los clientes para reservas DHCP

Se obtuvieron directamente las direcciones MAC de las interfaces enp2s0 de ambos clientes. Estas direcciones se utilizarán para conservar las IP 192.168.1.2 y 192.168.1.3 mediante reservas DHCP, mientras que el resto del rango dinámico quedará disponible para equipos adicionales.

Equipo	MAC	Dirección reservada
cliente1.redesi.test	0c:97:89:3b:00:00	192.168.1.2
cliente2.redesi.test	0c:01:6e:2b:00:00	192.168.1.3

Figura 5

Direcciones MAC obtenidas de DebianClient1 y DebianClient2.



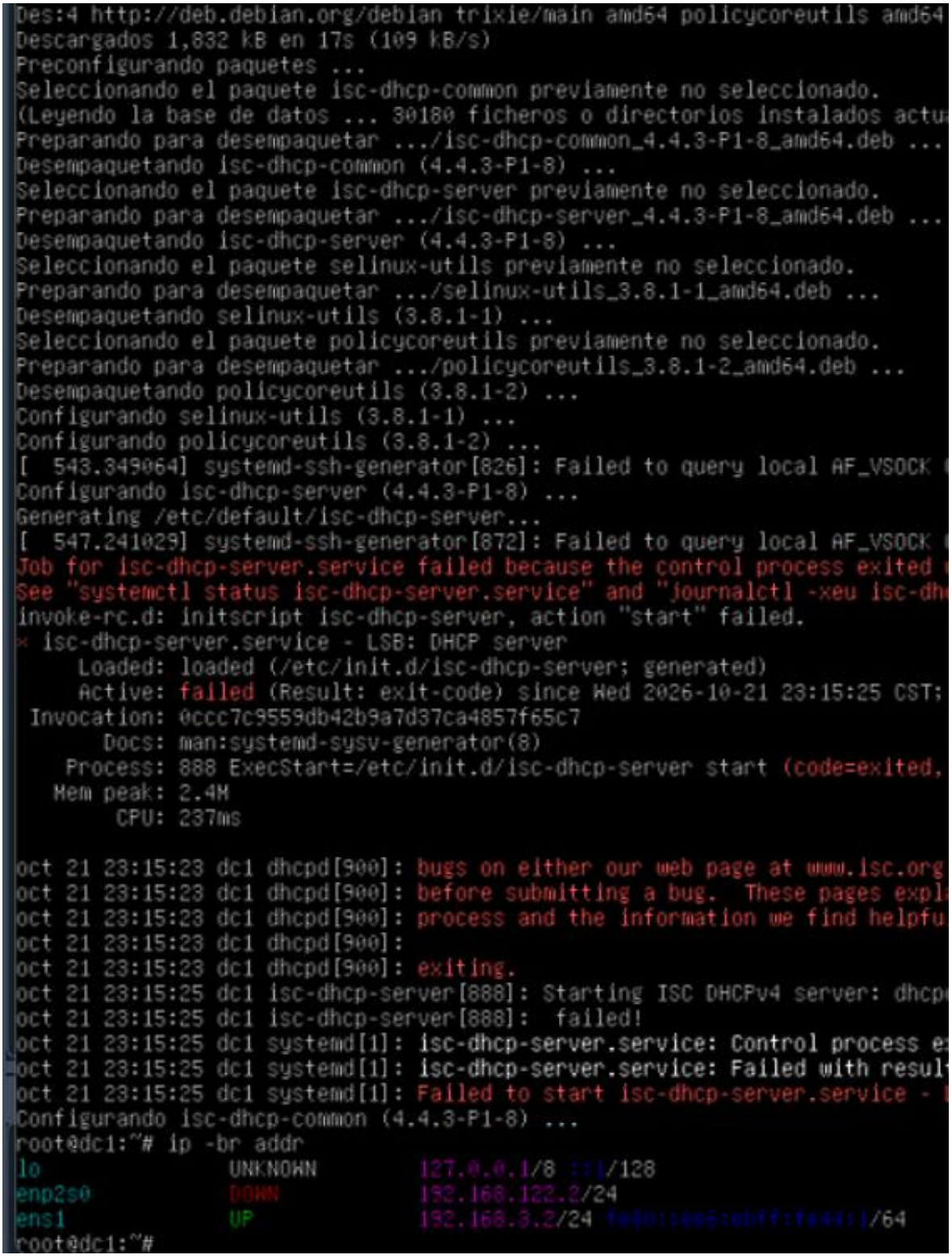
Nota. Las direcciones fueron consultadas directamente en /sys/class/net/enp2s0/address y serán utilizadas en las reservas del servidor DHCP.

Estado actual de la instalación DHCP

En dc1 se instaló el paquete isc-dhcp-server. Durante la instalación, el servicio no pudo iniciar porque todavía no existía una configuración DHCP completa, comportamiento esperado en esta etapa. Se verificó que ens1 está activa con 192.168.3.2/24 y se configuró INTERFACESv4="ens1" en /etc/default/isc-dhcp-server para que el servicio escuche en la interfaz interna de la LAN de servidores.

Figura 6

Preparación del servicio ISC DHCP Server en dc1.



Nota. La evidencia muestra la instalación del paquete, el fallo inicial del servicio antes de completar dhcpd.conf, la IP 192.168.3.2/24 en ens1 y la selección de ens1 como interfaz IPv4 del servicio.

Al cierre de esta actualización, DHCP se encuentra en implementación. Aún debe completarse /etc/dhcp/dhcpd.conf con el pool y las reservas, validar la sintaxis con dhcpd -t, iniciar el servicio, agregar ip helper-address 192.168.3.2 en FastEthernet0/0 de R1 y cambiar los clientes de direccionamiento manual a automático para ejecutar la prueba final. Por esta razón, el servicio no se marca todavía como completado.

Finalización y validación de DHCP IPv4

Actualización posterior. La sección anterior documenta el estado intermedio de la instalación. Posteriormente se completó la configuración del servicio DHCP en dc1, se configuró R1 como DHCP Relay, se migraron DebianClient1 y DebianClient2 de direccionamiento manual a automático y se ejecutaron pruebas de concesión, DNS, conectividad a Internet y resolución de usuarios del dominio. Por lo tanto, DHCP IPv4 queda marcado como completado en el estado consolidado del proyecto.

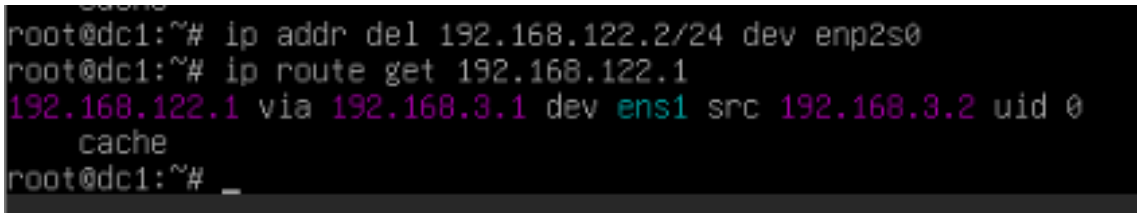
Corrección del acceso SSH y de la ruta de retorno de dc1

Durante la finalización del servicio se detectó que el acceso SSH publicado hacia dc1 por el puerto TCP 2223 dejaba de responder, mientras que los puertos 2222 y 2224 de los clientes sí funcionaban. Las pruebas desde la GNS3 VM confirmaron conectividad ICMP con 192.168.122.100 y alcance de los puertos de los clientes, aislando la incidencia al retorno de tráfico de dc1.

La consulta ip route get 192.168.122.1 en dc1 mostró que Linux intentaba responder por enp2s0 utilizando la dirección auxiliar 192.168.122.2. Esta interfaz ya no formaba parte del diseño final y generaba un retorno asimétrico. Se eliminó temporalmente la dirección con ip addr del 192.168.122.2/24 dev enp2s0 y se comprobó que la ruta pasó a utilizar 192.168.3.1 por ens1. Inmediatamente el puerto 2223 volvió a responder.

Figura 7

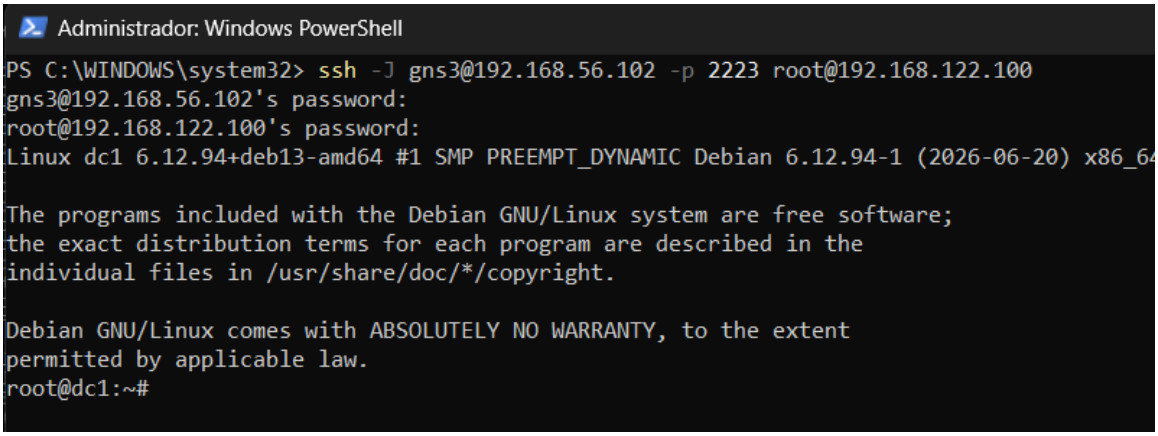
Corrección de la ruta de retorno de dc1.



Nota. Después de retirar 192.168.122.2/24 de enp2s0, dc1 selecciona 192.168.3.1 por ens1 como siguiente salto para llegar a 192.168.122.1.

Figura 8

Restablecimiento del acceso SSH remoto a dc1.



Nota. La sesión SSH desde Windows vuelve a ingresar correctamente como root a través del salto por la GNS3 VM y la publicación TCP 2223 de R1.

Configuración permanente de red y SSH en dc1

Para evitar que la incidencia reapareciera después de un reinicio, se respaldó /etc/network/interfaces y se eliminó de forma permanente el bloque de enp2s0. dc1 quedó únicamente con ens1 = 192.168.3.2/24, gateway 192.168.3.1 y la ruta hacia 192.168.1.0/24 por R2. La interfaz enp2s0 permanece sin dirección IPv4.

```
        source /etc/network/interfaces.d/*


auto lo

iface lo inet loopback


# Red interna del proyecto

auto ens1

iface ens1 inet static

    address 192.168.3.2/24

    gateway 192.168.3.1

    post-up ip route replace 192.168.1.0/24 via 192.168.3.1 dev ens1
```

Figura 9

Configuración definitiva de interfaces y ruta de dc1.

```
root@dc1:~# cat /etc/network/interfaces
source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback

# Red interna del proyecto
auto ens1
iface ens1 inet static
    address 192.168.3.2/24
    gateway 192.168.3.1
    post-up ip route replace 192.168.1.0/24 via 192.168.3.1 dev ens1
root@dc1:~# ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp2s0           DOWN
ens1             UP             192.168.3.2/24 fe80::ee6:ebff:fe44:1/64
root@dc1:~# ip route get 192.168.122.1
192.168.122.1 via 192.168.3.1 dev ens1 src 192.168.3.2 uid 0
    cache
```

Nota. ens1 conserva 192.168.3.2/24; enp2s0 queda DOWN sin IPv4 y la ruta a 192.168.122.1 utiliza 192.168.3.1 por ens1.

Debido a que el proyecto es un laboratorio que debe poder ser administrado por quien lo importe, se habilitó de forma permanente el acceso SSH de root por contraseña mediante

/etc/ssh/sshd_config.d/99-laboratorio.conf. Se validó la sintaxis con sshd -t, se reinició ssh.service y se comprobó el inicio de sesión remoto.

```
        PermitRootLogin yes

PasswordAuthentication yes
```

Configuración definitiva del servidor DHCP en dc1

El servidor ISC DHCP quedó configurado para atender la LAN remota 192.168.1.0/24 mediante el relay de R1. Se mantuvo un pool dinámico para equipos adicionales y se definieron reservas por MAC para conservar las direcciones de cliente1 y cliente2 utilizadas por Active Directory y las pruebas del proyecto.

```
        authoritative;
ddns-update-style none;

default-lease-time 600;
max-lease-time 7200;

option domain-name "redesi.test";
option domain-name-servers 192.168.3.2;

subnet 192.168.3.0 netmask 255.255.255.0 {
}

subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.150;
    option routers 192.168.1.1;
    option subnet-mask 255.255.255.0;
    option broadcast-address 192.168.1.255;
    option domain-name "redesi.test";
    option domain-name-servers 192.168.3.2;
}

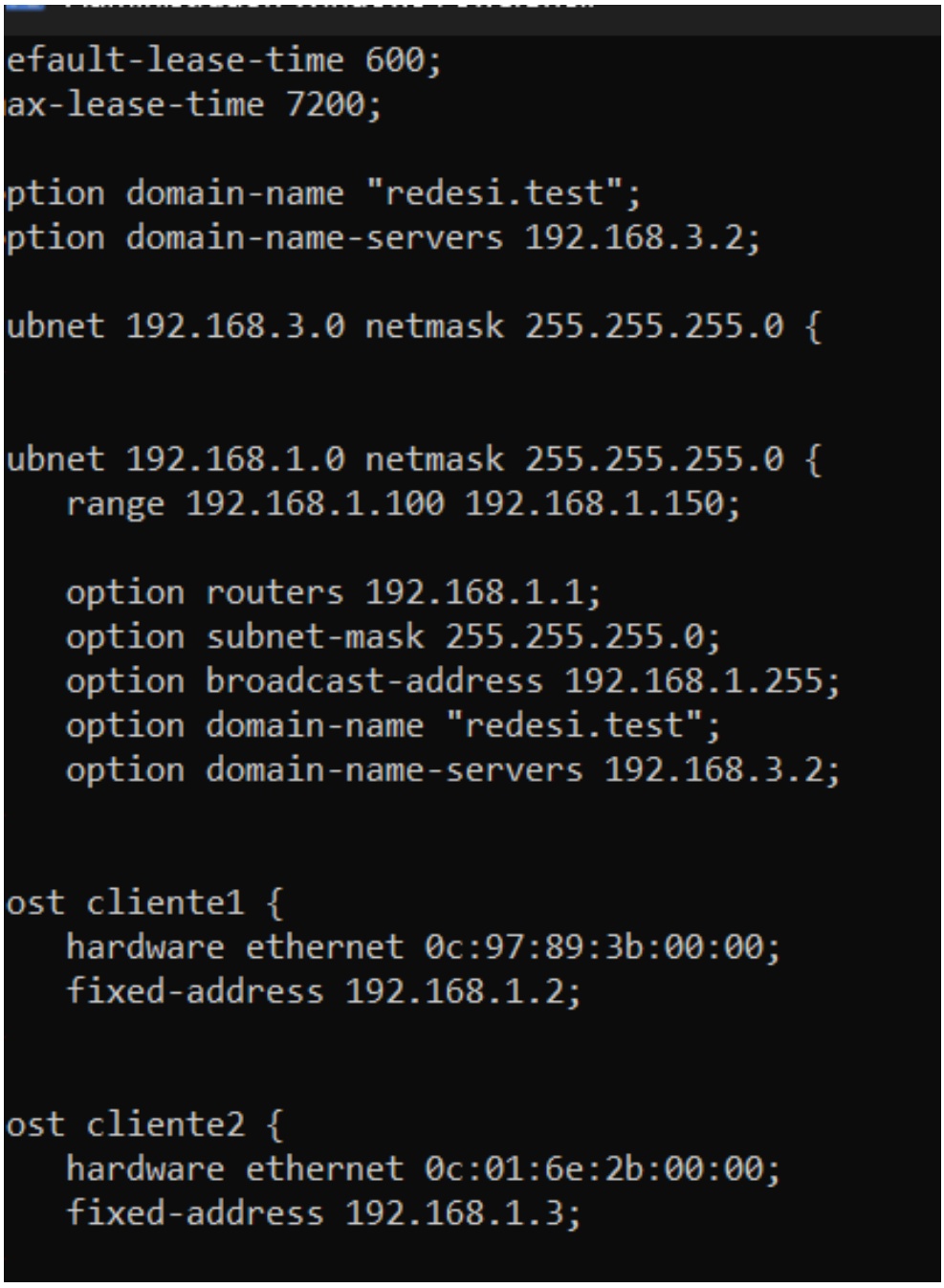
host cliente1 {
    hardware ethernet 0c:97:89:3b:00:00;
    fixed-address 192.168.1.2;
}

host cliente2 {
    hardware ethernet 0c:01:6e:2b:00:00;
    fixed-address 192.168.1.3;
}
```

En /etc/default/isc-dhcp-server se fijó INTERFACESv4="ens1" e INTERFACESv6="". La sintaxis se validó con dhcpd -t -cf /etc/dhcp/dhcpd.conf. Después del reinicio, isc-dhcp-server quedó active (running) y dhcpd escuchando en UDP/67.

Figura 10

Configuración final de dhcpd.conf.



```
efault-lease-time 600;
ax-lease-time 7200;

ption domain-name "redesi.test";
ption domain-name-servers 192.168.3.2;

ubnet 192.168.3.0 netmask 255.255.255.0 {


ubnet 192.168.1.0 netmask 255.255.255.0 {
   range 192.168.1.100 192.168.1.150;

   option routers 192.168.1.1;
   option subnet-mask 255.255.255.0;
   option broadcast-address 192.168.1.255;
   option domain-name "redesi.test";
   option domain-name-servers 192.168.3.2;

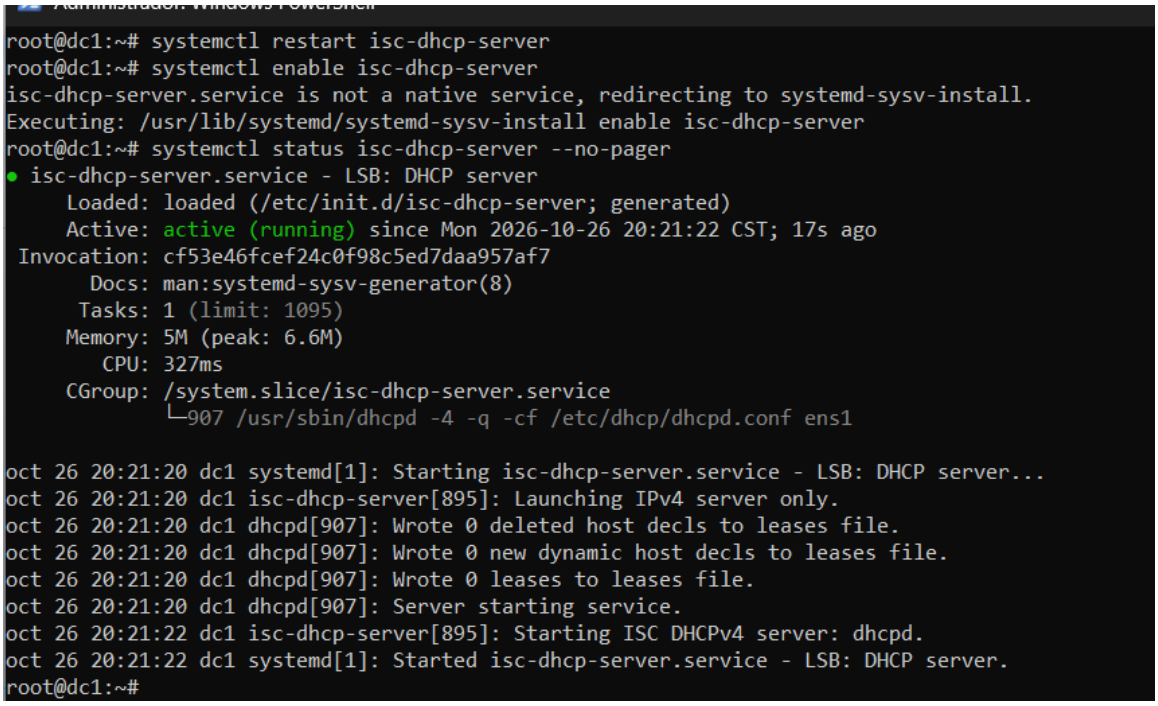

ost cliente1 {
   hardware ethernet 0c:97:89:3b:00:00;
   fixed-address 192.168.1.2;


ost cliente2 {
   hardware ethernet 0c:01:6e:2b:00:00;
   fixed-address 192.168.1.3;
```

Nota. El archivo contiene el pool 192.168.1.100–192.168.1.150 y las reservas 192.168.1.2 y 192.168.1.3 por MAC.

Figura 11

Servicio ISC DHCP Server activo en dc1.



Nota. systemctl confirma isc-dhcp-server active (running); el proceso dhcpd se inicia sobre ens1.

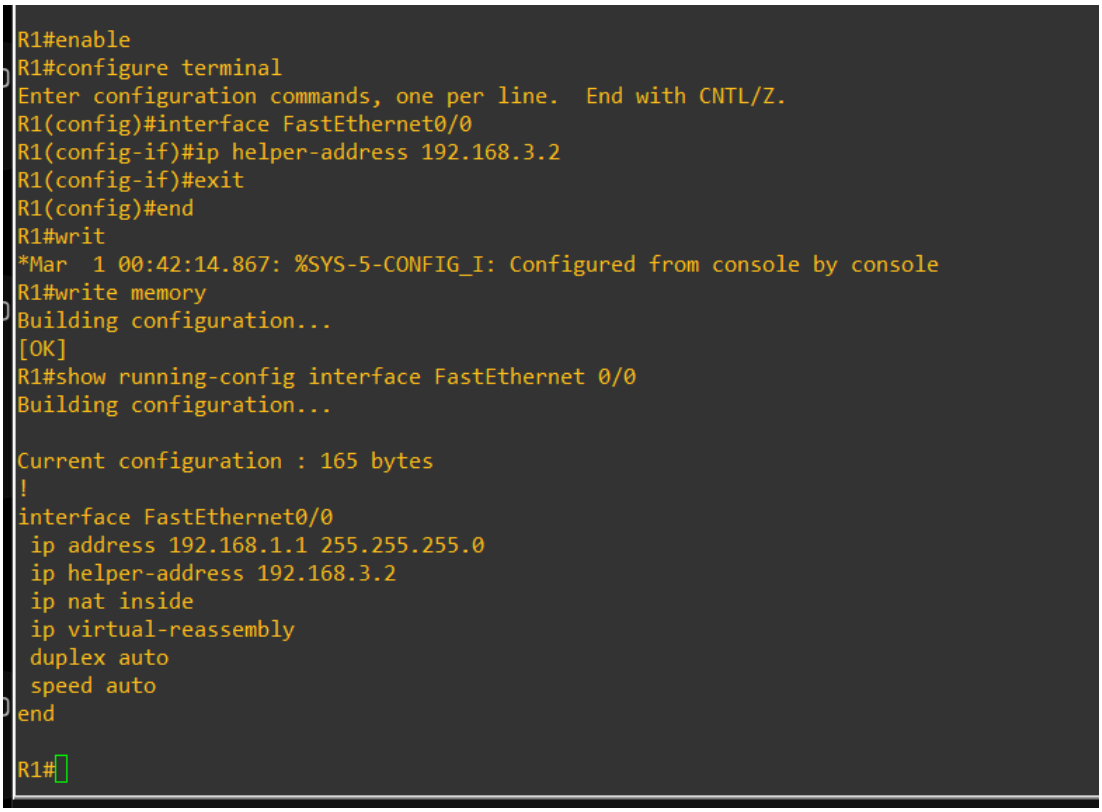
Configuración de R1 como DHCP Relay

Como los clientes se encuentran en 192.168.1.0/24 y dc1 en 192.168.3.0/24, los broadcasts DHCP no atraviesan los routers de forma directa. Se configuró FastEthernet0/0 de R1 con ip helper-address 192.168.3.2 para reenviar las solicitudes hacia dc1. La configuración se guardó con write memory.

```
       interface FastEthernet0/0
 ip address 192.168.1.1 255.255.255.0
 ip helper-address 192.168.3.2
 ip nat inside
```

Figura 12

DHCP Relay configurado en R1.



Nota. FastEthernet0/0 mantiene 192.168.1.1/24, ip nat inside y agrega ip helper-address 192.168.3.2.

Migración de DebianClient1 y DebianClient2 a DHCP

En ambos clientes se utilizó NetworkManager. El perfil Wired connection 1 se cambió a ipv4.method auto y se retiraron los valores manuales de addresses, gateway y dns. En cliente1 fue necesario borrar primero el gateway antes de limpiar la dirección manual, debido a la validación de NetworkManager. Después de reactivar la conexión, las reservas devolvieron las mismas direcciones previamente utilizadas.

```
        nmcli con mod "Wired connection 1" ipv4.method auto

nmcli con mod "Wired connection 1" ipv4.gateway ""

nmcli con mod "Wired connection 1" ipv4.addresses ""

nmcli con mod "Wired connection 1" ipv4.dns ""

nmcli con down "Wired connection 1"

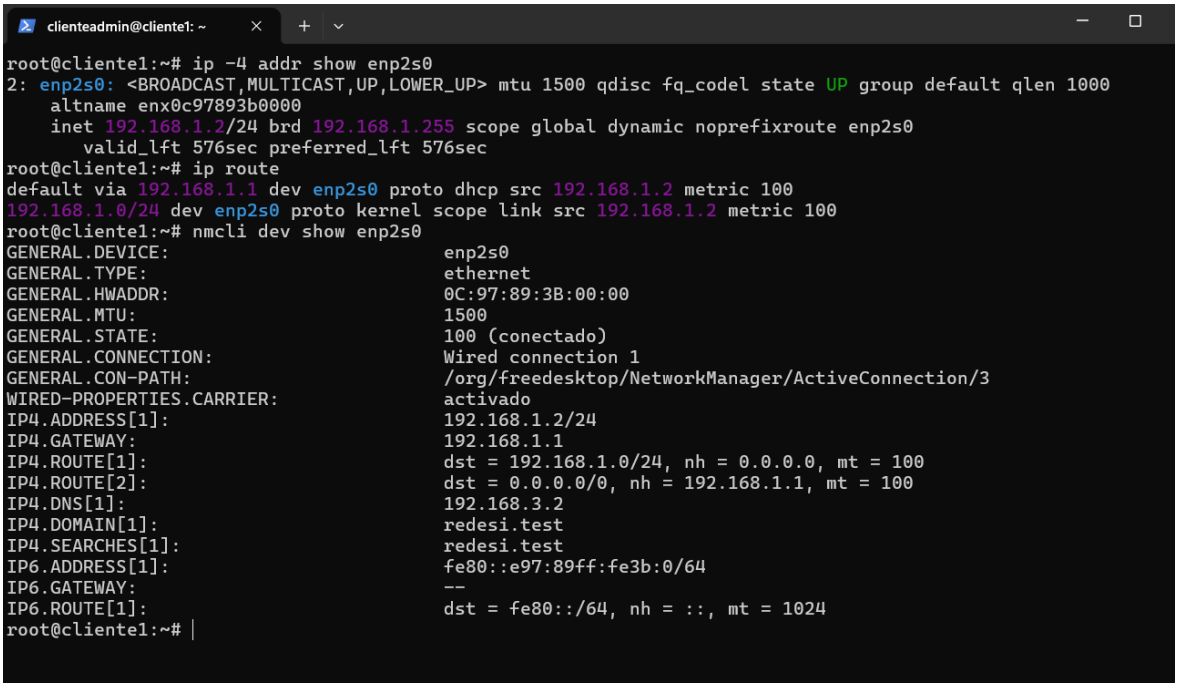
nmcli con up "Wired connection 1"
```

Resultado final de las concesiones:

Equipo	MAC	IP por DHCP	Parámetros entregados
cliente1	0c:97:89:3b:00:00	192.168.1.2/24	GW 192.168.1.1 · DNS 192.168.3.2 · redesi.test
cliente2	0c:01:6e:2b:00:00	192.168.1.3/24	GW 192.168.1.1 · DNS 192.168.3.2 · redesi.test

Figura 13

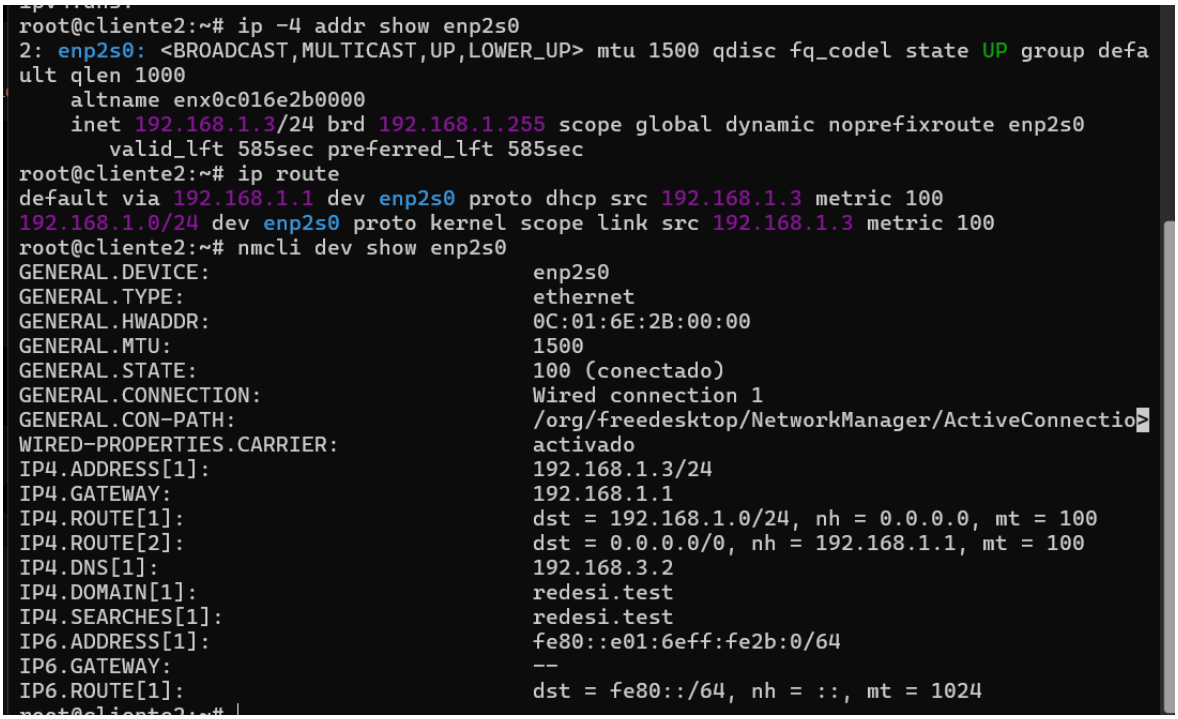
Parámetros DHCP obtenidos por DebianClient1.



Nota. La interfaz enp2s0 recibe dinámicamente 192.168.1.2/24; la ruta muestra proto dhcp, gateway 192.168.1.1, DNS 192.168.3.2 y dominio redesi.test.

Figura 14

Parámetros DHCP obtenidos por DebianClient2.



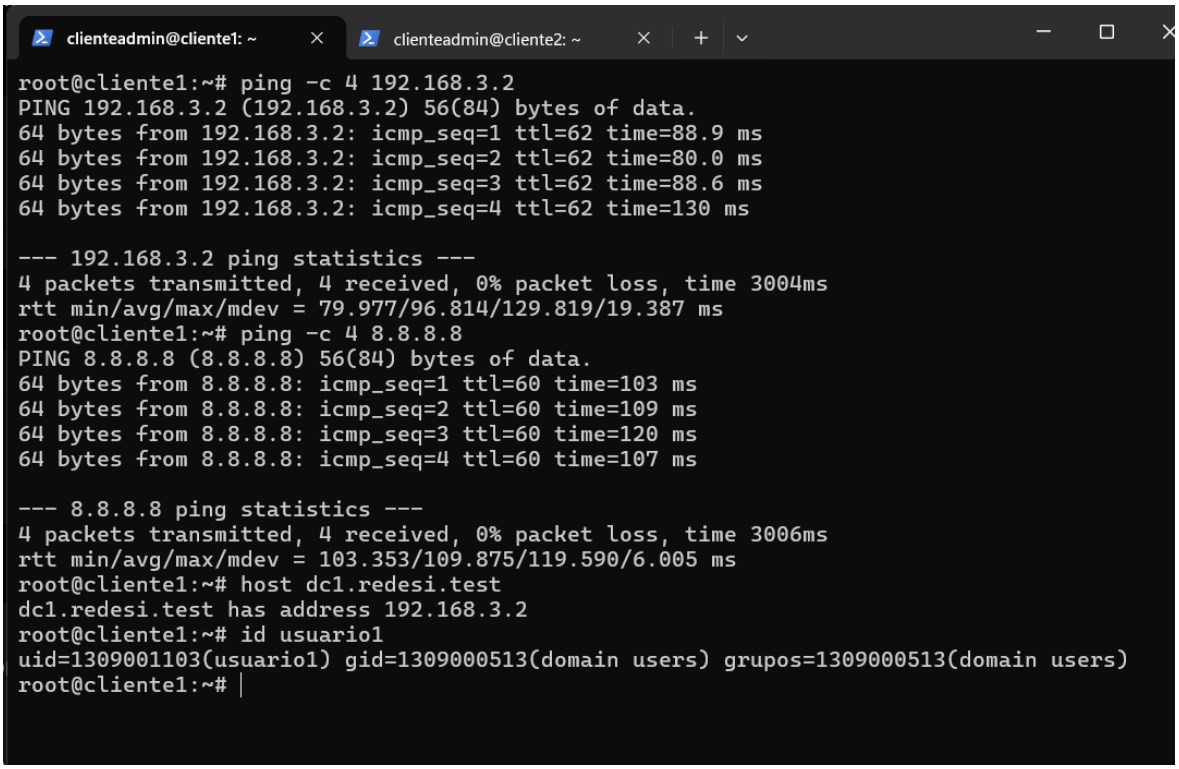
Nota. La interfaz enp2s0 recibe dinámicamente 192.168.1.3/24; NetworkManager confirma gateway 192.168.1.1, DNS 192.168.3.2 y dominio redesi.test.

Pruebas integrales de conectividad, DNS y Active Directory

Después de obtener las concesiones, ambos clientes conservaron conectividad con dc1 y acceso a Internet. La resolución de dc1.redesi.test devolvió 192.168.3.2 y SSSD continuó resolviendo los usuarios del dominio: usuario1 en cliente1 y usuario2 en cliente2. Esto demuestra que el cambio de direccionamiento manual a DHCP no afectó la integración con Samba AD, DNS ni Kerberos.

Figura 15

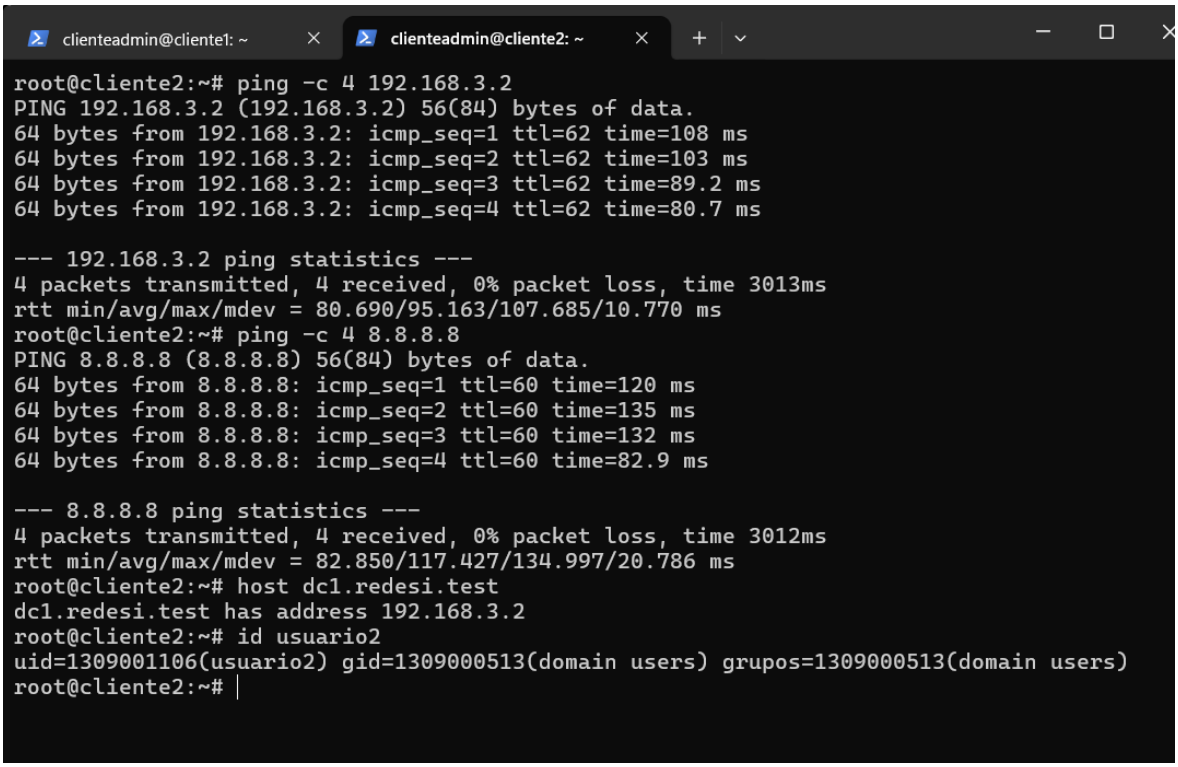
Pruebas finales desde DebianClient1.



Nota. Cliente1 alcanza 192.168.3.2 y 8.8.8.8 sin pérdida, resuelve dc1.redesi.test y reconoce usuario1 como miembro de domain users.

Figura 16

Pruebas finales desde DebianClient2.



Nota. Cliente2 alcanza 192.168.3.2 y 8.8.8.8 sin pérdida, resuelve dc1.redesi.test y reconoce usuario2 como miembro de domain users.

Evidencia del proceso DHCP en el servidor

Los registros de isc-dhcp-server muestran el ciclo completo DHCPDISCOVER → DHCPOFFER → DHCPREQUEST → DHCPACK para ambos clientes a través del relay 192.168.1.1. Antes de la concesión correcta, cada cliente intentó renovar una dirección antigua 192.168.122.160; el servidor respondió DHCPNAK por pertenecer a una red incorrecta y, a continuación, asignó la reserva válida. Este comportamiento confirma que el servidor rechazó la concesión obsoleta y entregó correctamente las direcciones de la LAN de clientes.

Pruebas de verificación de DHCP IPv4

Al finalizar la configuración de ISC DHCP Server, el relay en R1 y la migración de los clientes a direccionamiento automático, se realizaron pruebas de sintaxis, servicio, concesión, opciones DHCP, conectividad, DNS y continuidad de Active Directory.

Elemento comprobado	Prueba / comando	Resultado observado	Estado
Sintaxis del servidor DHCP	dhcpd -t -cf /etc/dhcp/dhcpd.conf	Configuración validada sin errores.	**APROBADO**
Estado del servicio	systemctl status isc-dhcp-server --no-pager	active (running); dhcpd sobre ens1 y UDP/67.	**APROBADO**
Relay en R1	show running-config interface FastEthernet0/0	ip helper-address 192.168.3.2 presente y guardado.	**APROBADO**
Reserva Cliente1	ip -4 addr / ip route / nmcli dev show enp2s0	192.168.1.2/24, GW 192.168.1.1, DNS 192.168.3.2, redesi.test.	**APROBADO**
Reserva Cliente2	ip -4 addr / ip route / nmcli dev show enp2s0	192.168.1.3/24, GW 192.168.1.1, DNS 192.168.3.2, redesi.test.	**APROBADO**
Ciclo DHCP Cliente1	journalctl -u isc-dhcp-server	DISCOVER → OFFER 192.168.1.2 → REQUEST → ACK vía 192.168.1.1.	**APROBADO**
Ciclo DHCP Cliente2	journalctl -u isc-dhcp-server	DISCOVER → OFFER 192.168.1.3 → REQUEST → ACK vía 192.168.1.1.	**APROBADO**
Conectividad y DNS	ping dc1 / ping 8.8.8.8 / host dc1.redesi.test	0 % de pérdida; DNS resuelve 192.168.3.2.	**APROBADO**
Continuidad de AD	id usuario1 / id usuario2	SSSD sigue resolviendo domain users después del cambio a DHCP.	**APROBADO**

```
        Cliente1 (MAC 0c:97:89:3b:00:00)

DHCPDISCOVER ... via 192.168.1.1

DHCPOFFER on 192.168.1.2 ... via 192.168.1.1

DHCPREQUEST for 192.168.1.2 ... via 192.168.1.1

DHCPACK on 192.168.1.2 ... via 192.168.1.1


Cliente2 (MAC 0c:01:6e:2b:00:00)

DHCPDISCOVER ... via 192.168.1.1

DHCPOFFER on 192.168.1.3 ... via 192.168.1.1

DHCPREQUEST for 192.168.1.3 ... via 192.168.1.1

DHCPACK on 192.168.1.3 ... via 192.168.1.1
```

Resultado final: el servicio DHCP IPv4 se considera completado. dc1 entrega direcciones, gateway, DNS y dominio; R1 realiza el relay entre redes; cliente1 conserva 192.168.1.2 y cliente2

192.168.1.3 mediante reservas por MAC; y ambos equipos mantienen acceso a Internet y autenticación de dominio.

Implementación y validación de FTP seguro mediante TLS (FTPS)

Actualización posterior. Después de completar DHCP IPv4 se implementó el servicio FTP seguro solicitado para el proyecto. Se utilizó vsftpd en dc1.redesi.test (192.168.3.2) y FTPS explícito sobre TCP/21, manteniendo el acceso anónimo deshabilitado y obligando el uso de TLS para autenticación y transferencia de datos. El servicio se validó desde DebianClient1 y DebianClient2 mediante pruebas de conexión, cifrado, autenticación, listado, descarga, subida y transferencia entre clientes.

Parámetro	Configuración final
Servidor	dc1.redesi.test / 192.168.3.2
Software	vsftpd 3.0.5
Protocolo	FTPS explícito (FTP + TLS)
Puerto de control	TCP/21
Puertos pasivos	TCP/40000–40010
Usuario de prueba	ftpuser
Directorio del usuario	/home/ftpuser/ftp
Directorio de transferencia	/home/ftpuser/ftp/files
Acceso anónimo	Deshabilitado
Cifrado validado	TLS 1.3 · TLS_AES_256_GCM_SHA384

Cuenta, directorios y permisos del servicio

Se creó la cuenta local ftpuser exclusivamente para el servicio FTPS. Su directorio de inicio se fijó en /home/ftpuser/ftp. La raíz del chroot pertenece a root:root y el subdirectorio files pertenece a ftpuser:ftpuser, permitiendo transferencias sin hacer escribible la raíz del entorno aislado.

```
        useradd -m -s /bin/bash ftpuser
passwd ftpuser
```

```
usermod -d /home/ftpuser/ftp ftpuser
mkdir -p /home/ftpuser/ftp/files
chown root:root /home/ftpuser/ftp
chmod 755 /home/ftpuser/ftp
chown ftpuser:ftpuser /home/ftpuser/ftp/files
```

Como archivo de prueba inicial se creó prueba_servidor.txt con el texto “Servidor FTPS dc1 - Proyecto Redes I”. Este archivo permitió comprobar posteriormente la descarga desde los clientes.

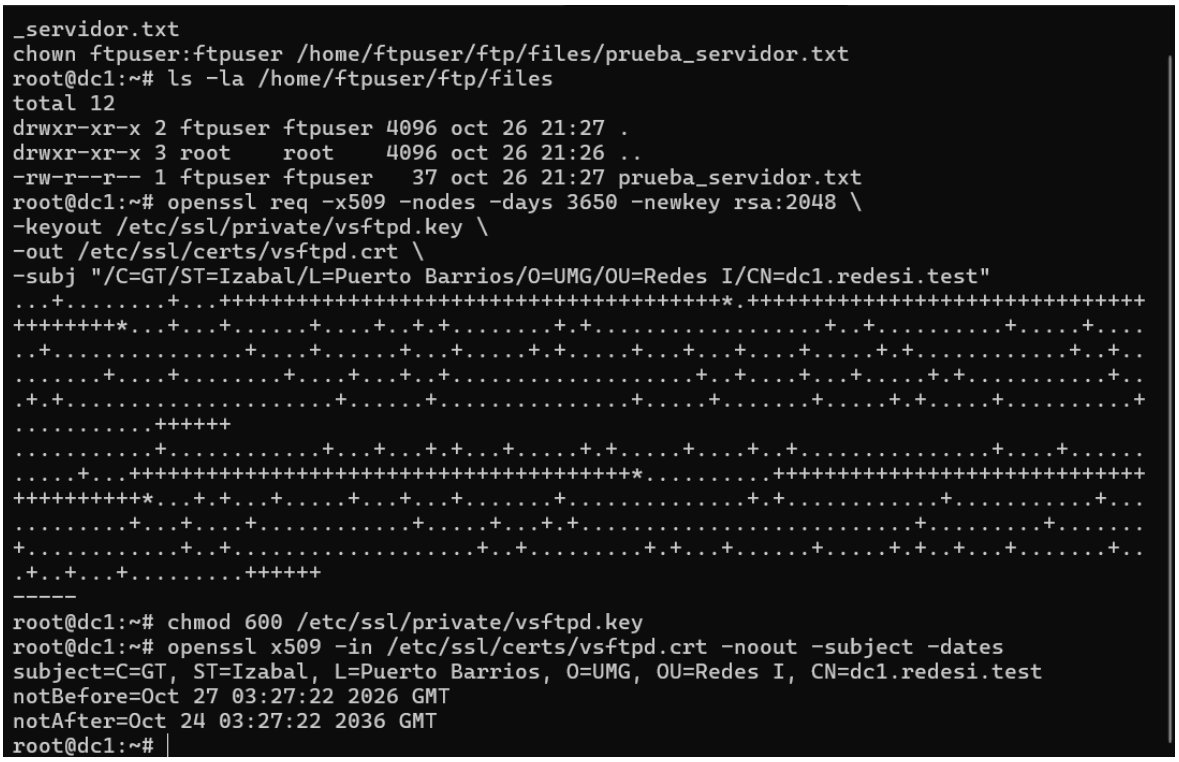
Certificado TLS para dc1.redesi.test

Se generó un certificado X.509 autofirmado para el laboratorio, con clave RSA de 2048 bits y CN=dc1.redesi.test. La clave privada se protegió con permisos 600. El certificado tiene vigencia desde el 27 de octubre de 2026 hasta el 24 de octubre de 2036 (fechas mostradas por OpenSSL en GMT). Al ser autofirmado, los clientes muestran verify return code 18; este aviso es esperado en el entorno académico y no impidió la negociación TLS.

```
        openssl req -x509 -nodes -days 3650 -newkey rsa:2048 \
-keyout /etc/ssl/private/vsftpd.key \
-out /etc/ssl/certs/vsftpd.crt \
-subj "/C=GT/ST=Izabal/L=Puerto Barrios/O=UMG/OU=Redes I/CN=dc1.redesi.test"
chmod 600 /etc/ssl/private/vsftpd.key
openssl x509 -in /etc/ssl/certs/vsftpd.crt -noout -subject -dates
```

Figura 17

Generación y verificación del certificado TLS para FTPS.



Nota. OpenSSL confirma el sujeto C=GT, ST=Izabal, L=Puerto Barrios, O=UMG, OU=Redes I y CN=dc1.redesi.test, además de las fechas de validez del certificado.

Configuración final de vsftpd

La configuración final de /etc/vsftpd.conf limita el servicio a 192.168.3.2:21, deshabilita usuarios anónimos, permite usuarios locales, encierra al usuario en su directorio mediante chroot, habilita escritura en la subcarpeta autorizada, define el rango pasivo 40000–40010 y obliga TLS tanto para el inicio de sesión como para el canal de datos.

```
         listen=YES
listen_ipv6=NO
listen_address=192.168.3.2
listen_port=21

anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022

chroot_local_user=YES
local_root=/home/ftpuser/ftp
```

```
dirmessage_enable=YES
use_localtime=YES

xferlog_enable=YES
xferlog_std_format=NO
log_ftp_protocol=YES

pasv_enable=YES
pasv_address=192.168.3.2
pasv_min_port=40000
pasv_max_port=40010

ssl_enable=YES
force_local_logins_ssl=YES
force_local_data_ssl=YES
require_ssl_reuse=NO

rsa_cert_file=/etc/ssl/certs/vsftpd.crt
rsa_private_key_file=/etc/ssl/private/vsftpd.key

pam_service_name=vsftpd
secure_chroot_dir=/var/run/vsftpd/empty
```

Verificación del servicio y del puerto TCP/21

Después de corregir un primer archivo de configuración que había quedado incompleto durante el pegado, vsftpd inició correctamente. systemctl confirmó el estado active (running) y ss -tlnp verificó que el proceso escucha exclusivamente en 192.168.3.2:21.

```
        systemctl status vsftpd --no-pager
Active: active (running)
Main PID: 1477 (vsftpd)

ss -tlnp
LISTEN ... 192.168.3.2:21 ... users:(("vsftpd",pid=1477,fd=3))
```

Prueba de negociación TLS desde DebianClient1

Desde cliente1 se ejecutó OpenSSL contra dc1.redesi.test:21 utilizando STARTTLS para FTP. La negociación presentó el certificado esperado y estableció TLS 1.3 con el cifrado TLS_AES_256_GCM_SHA384. La respuesta 220 (vsFTPd 3.0.5) confirmó que el canal de control FTP quedó disponible después del handshake.

```
        openssl s_client -connect dc1.redesi.test:21 -starttls ftp -servername dc1.redesi.test

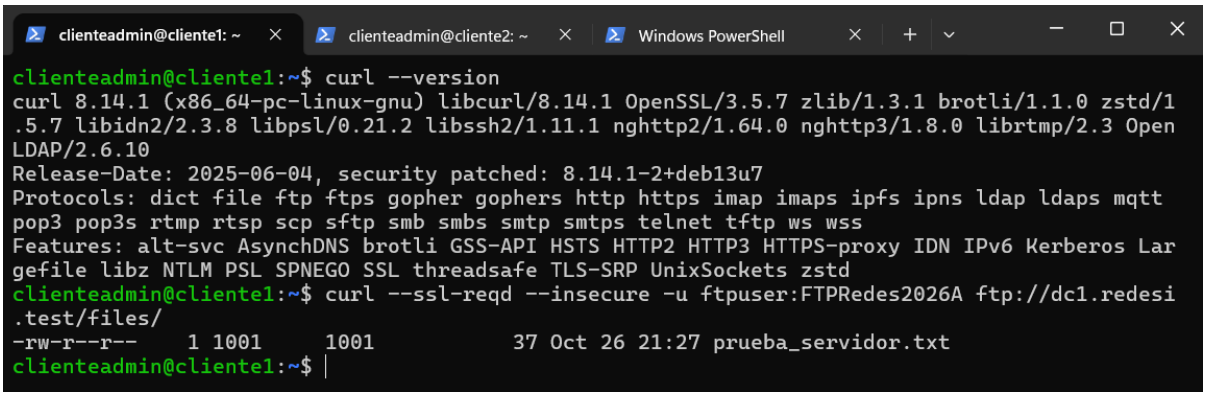
subject=C=GT, ST=Izabal, L=Puerto Barrios, O=UMG, OU=Redes I, CN=dc1.redesi.test
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Verify return code: 18 (self-signed certificate)
220 (vsFTPd 3.0.5)
```

Autenticación y listado seguro desde DebianClient1

Se instaló curl en cliente1 y se utilizó --ssl-reqd para exigir TLS. El primer listado autenticado con ftpuser mostró prueba_servidor.txt, confirmando que el cliente puede resolver dc1.redesi.test, autenticarse y abrir el canal de datos FTPS.

Figura 18

Autenticación FTPS y listado de archivos desde DebianClient1.



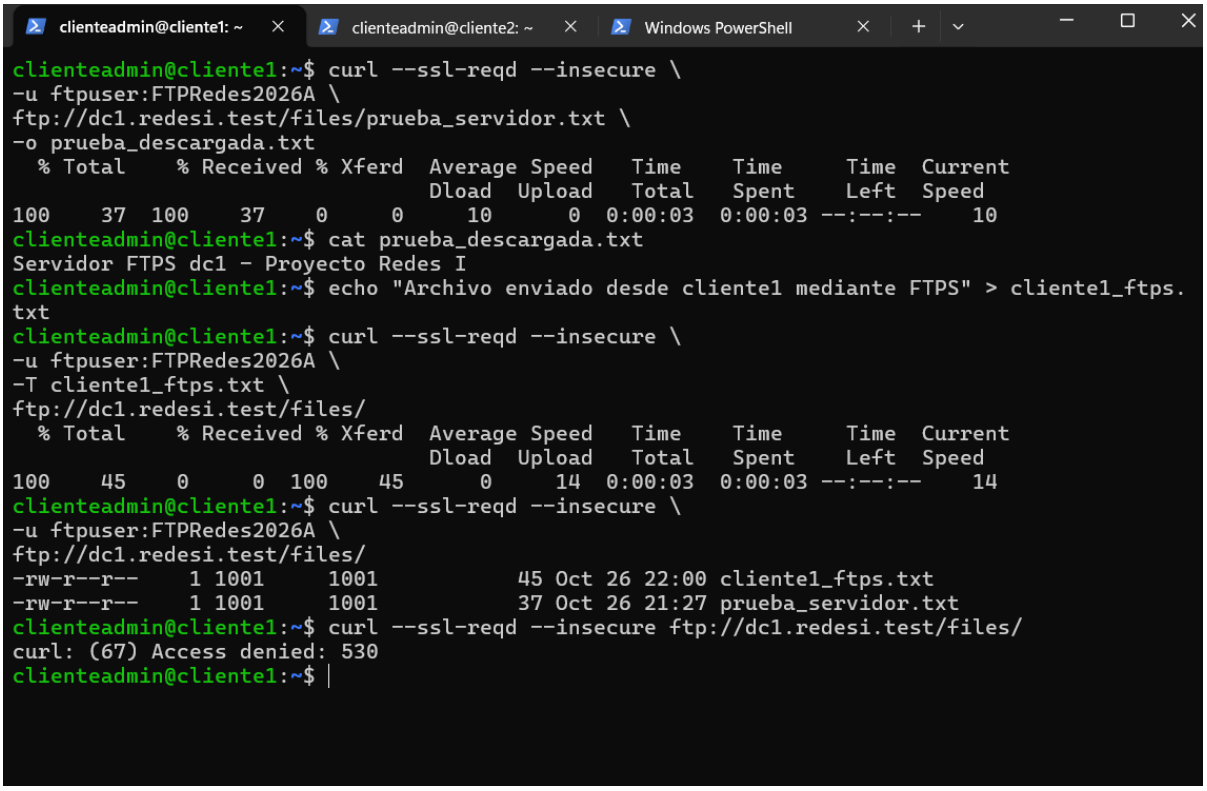
Nota. El listado autenticado mediante curl muestra prueba_servidor.txt dentro de /files/.

Descarga, subida y bloqueo del acceso anónimo

En cliente1 se descargó prueba_servidor.txt y se verificó su contenido. Después se creó cliente1_ftps.txt, se subió al servidor y un nuevo listado confirmó ambos archivos. Finalmente se repitió la consulta sin credenciales; vsftpd respondió 530 Access denied, demostrando que el acceso anónimo está deshabilitado.

Figura 19

Pruebas de descarga, subida y rechazo del acceso anónimo en DebianClient1.



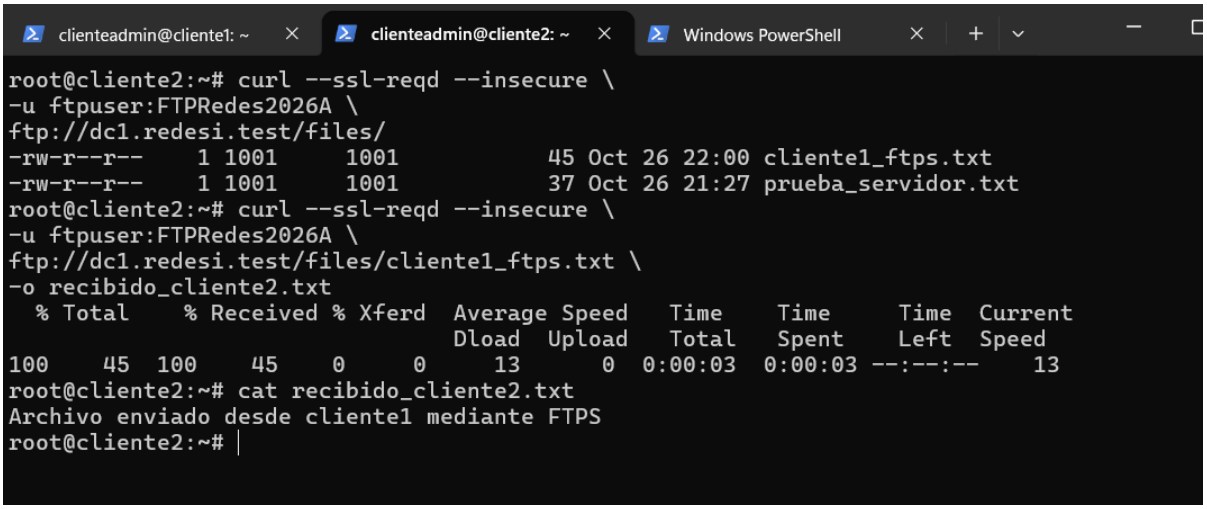
Nota. La evidencia muestra la descarga de prueba_servidor.txt, la subida de cliente1_ftps.txt, el listado de ambos archivos y el rechazo 530 Access denied cuando se intenta acceder sin credenciales.

Transferencia entre clientes mediante el servidor FTPS

Para comprobar el servicio desde una segunda máquina, cliente2 se autenticó con la misma cuenta FTPS, listó el directorio y descargó cliente1_ftps.txt, archivo que había sido creado y cargado previamente desde cliente1. El comando cat confirmó exactamente el contenido “Archivo enviado desde cliente1 mediante FTPS”. Esta prueba demuestra el flujo Cliente1 → dc1 → Cliente2 a través de FTPS.

Figura 20

Descarga en DebianClient2 del archivo enviado previamente desde DebianClient1.



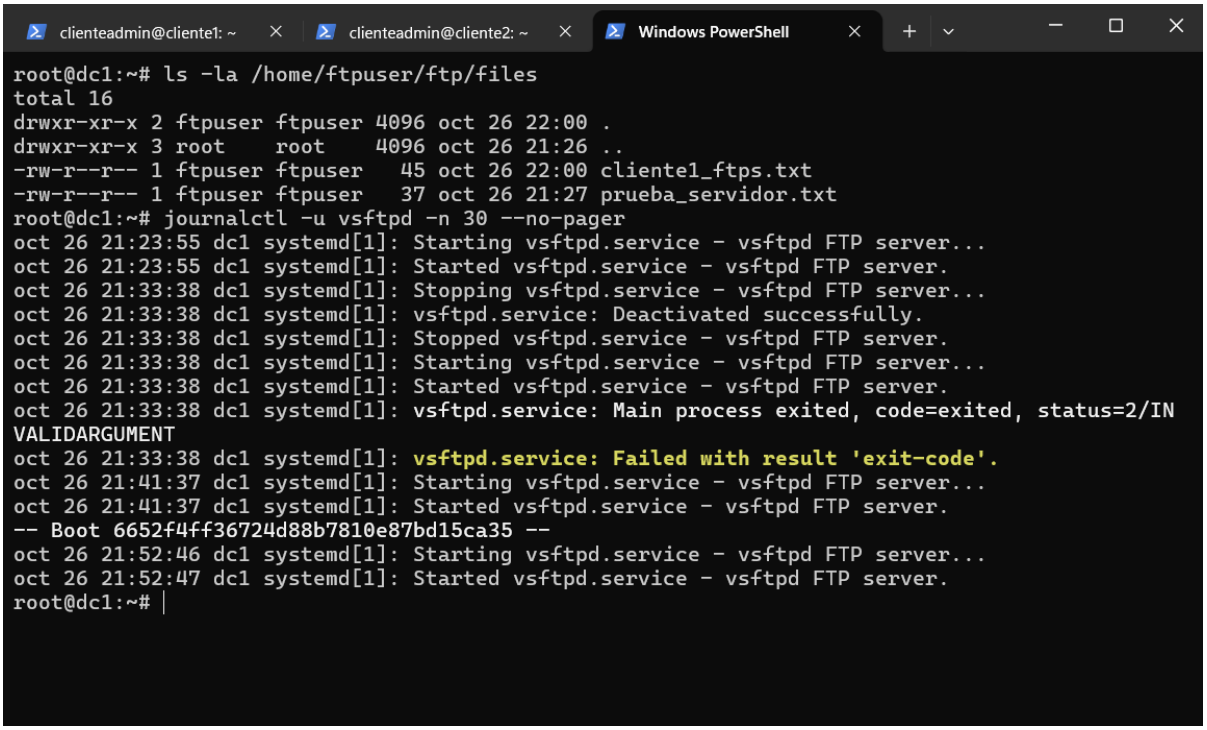
Nota. Cliente2 lista el contenido de /files/, descarga cliente1_ftps.txt y muestra correctamente el texto transferido desde cliente1.

Comprobación en el servidor y registros de auditoría

En dc1 se verificó que los dos archivos permanecen almacenados en /home/ftpuser/ftp/files y pertenecen a ftpuser. Además, /var/log/vsftpd.log registra las operaciones FTP detalladas, incluyendo autenticación, protección del canal de datos, modo pasivo y descarga.

Figura 21

Archivos almacenados en dc1 y estado del servicio vsftpd.



Nota. El servidor conserva cliente1_ftps.txt y prueba_servidor.txt. El historial del servicio también muestra que vsftpd quedó iniciado correctamente después de corregir la configuración.

```
        CONNECT: Client "192.168.1.3"
AUTH SSL
234 Proceed with negotiation.
USER ftpuser
PASS <password>
OK LOGIN: Client "192.168.1.3"
230 Login successful.
PBSZ 0
PROT P
200 PROT now Private.
EPSV
229 Entering Extended Passive Mode (|||40009|)
RETR cliente1_ftps.txt
OK DOWNLOAD: Client "192.168.1.3", "/files/cliente1_ftps.txt", 45 bytes
226 Transfer complete.
```

El registro PROT P / 200 PROT now Private confirma que el canal de datos se protegió; EPSV con puertos 40000 y 40009 demuestra el uso del rango pasivo configurado; OK LOGIN confirma autenticación y OK DOWNLOAD junto con 226 Transfer complete confirma la descarga completa de 45 bytes desde cliente2.

Pruebas de verificación del servicio FTPS

Elemento comprobado	Prueba / comando	Resultado observado	Conclusión
Servicio vsftpd	systemctl status vsftpd	active (running)	Servicio operativo
Puerto de control	ss -tlnp	192.168.3.2:21 LISTEN	TCP/21 disponible
Certificado X.509	openssl x509 ... -subject -dates	CN=dc1.redesi.test; vigencia 2026–2036	Certificado correcto
TLS del canal de control	openssl s_client -starttls ftp	TLS 1.3 / TLS_AES_256_GCM_SHA384	Cifrado negociado
Autenticación	curl --ssl-reqd -u ftpuser:...	Listado exitoso	Credenciales válidas
Acceso anónimo	curl --ssl-reqd sin -u	530 Access denied	Anónimo bloqueado
Descarga Cliente1	curl ... prueba_servidor.txt	37 bytes; contenido correcto	Descarga funcional
Subida Cliente1	curl -T cliente1_ftps.txt	45 bytes cargados	Escritura funcional
Listado posterior	curl ... /files/	Dos archivos visibles	Cambios persistentes
Descarga Cliente2	curl ... cliente1_ftps.txt	45 bytes; contenido correcto	Transferencia entre clientes
Canal de datos privado	vsftpd.log	PROT P / 200 PROT now Private	Datos protegidos por TLS
Modo pasivo	vsftpd.log	EPSV; puertos 40000 y 40009	Rango pasivo operativo
Auditoría de descarga	vsftpd.log	OK DOWNLOAD / 226 Transfer complete	Transferencia registrada

Elemento comprobado	Prueba / comando	Resultado observado	Conclusión
DNS MX de redesi.test	Completado	Prioridad 10 hacia dc1.redesi.test; validado desde servidor y cliente.	
SMTP + STARTTLS	Completado	TLS 1.3 y entrega Maildir validados desde cliente1 y cliente2.	
IMAP / IMAPS	Completado	Dovecot 993/TLS; lectura de buzones comprobada en ambos clientes.	
POP3 / POP3S	Completado	Dovecot 995/TLS; STAT, LIST y RETR comprobados.	

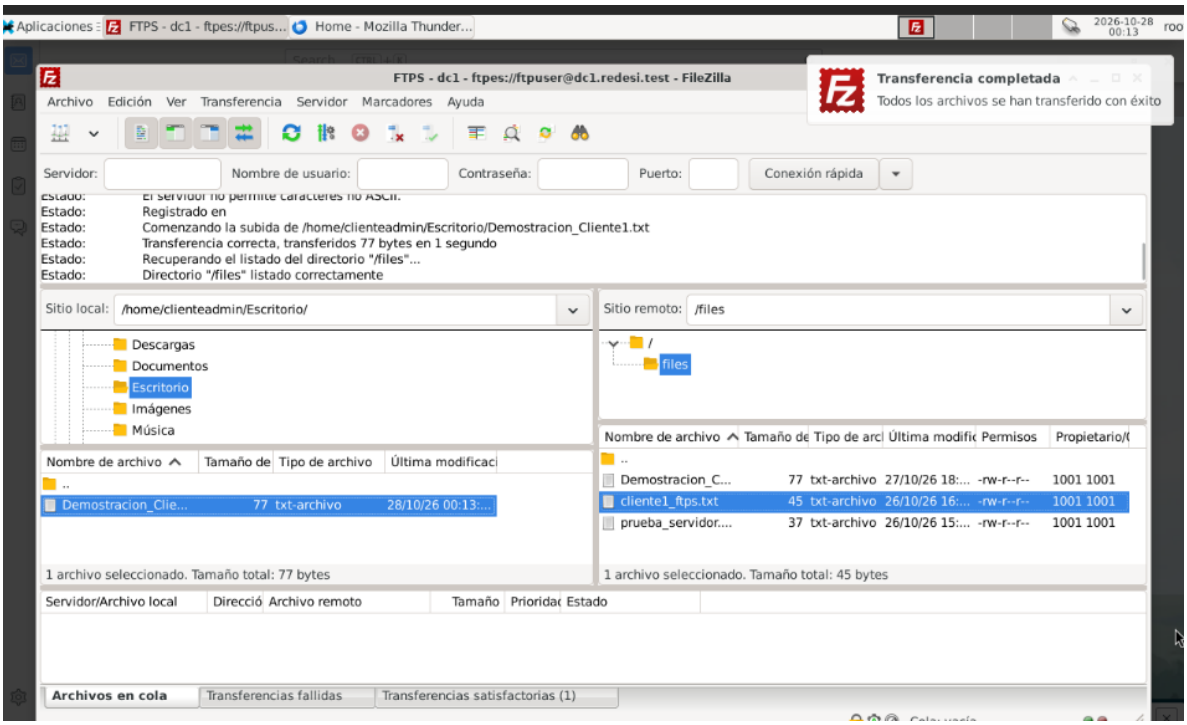
Demostración visual de FTPS con FileZilla

Como complemento a las pruebas por consola, FileZilla se instaló en los clientes Debian. El sitio se configuró contra dc1.redesi.test por TCP/21, utilizando FTP con TLS explícito, usuario ftpuser y la contraseña del laboratorio. Se aceptó el certificado autofirmado de dc1 únicamente dentro de este entorno académico.

Parámetro	Configuración / resultado
Servidor	dc1.redesi.test
Puerto	21
Protocolo	FTP - Protocolo de Transferencia de Archivos
Cifrado	Requerir FTP explícito sobre TLS
Usuario	ftpuser
Contraseña	FTPRedes2026A
Directorio remoto	/files

Figura 27

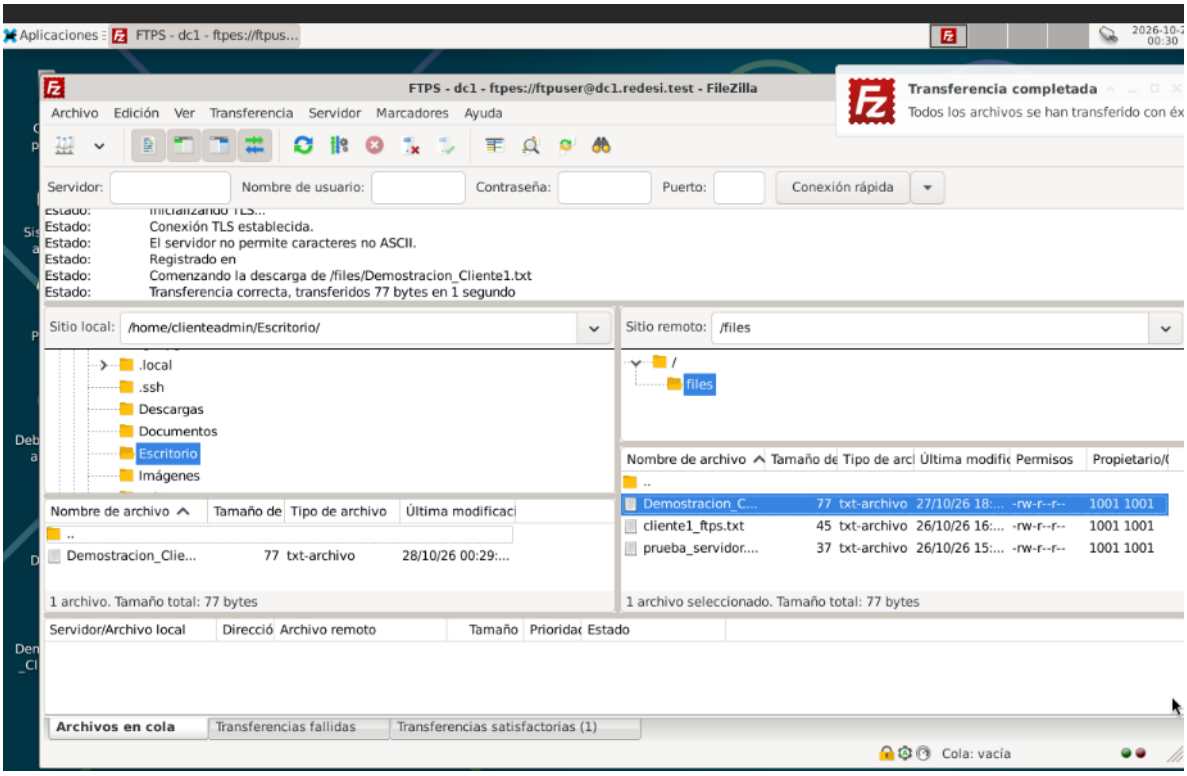
Carga de archivo desde DebianClient1 mediante FileZilla.



Nota. FileZilla informa “Transferencia completada” y el archivo Demostracion_Cliente1.txt aparece en /files junto con las evidencias previas.

Figura 28

Descarga del archivo de demostración desde el segundo cliente.



Nota. La descarga se completa correctamente y confirma de forma visual que ambos clientes pueden utilizar el servicio FTPS configurado en dc1.

Resultado final de FTPS

Resultado final: FTP seguro mediante TLS se considera COMPLETADO. dc1 ofrece FTPS explícito con TLS 1.3, autenticación obligatoria, acceso anónimo deshabilitado, chroot para el usuario ftpuser y modo pasivo 40000–40010. Cliente1 y Cliente2 pueden listar, descargar y transferir archivos mediante el servidor, y vsftpd.log deja evidencia del canal privado y de las transferencias.

Con AD, DNS, Kerberos, DHCP y FTPS completados, el último servicio principal pendiente del alcance actual es el sistema de correo electrónico mediante SMTP y IMAP y/o POP3, seguido de la prueba integral final y la consolidación de la documentación.

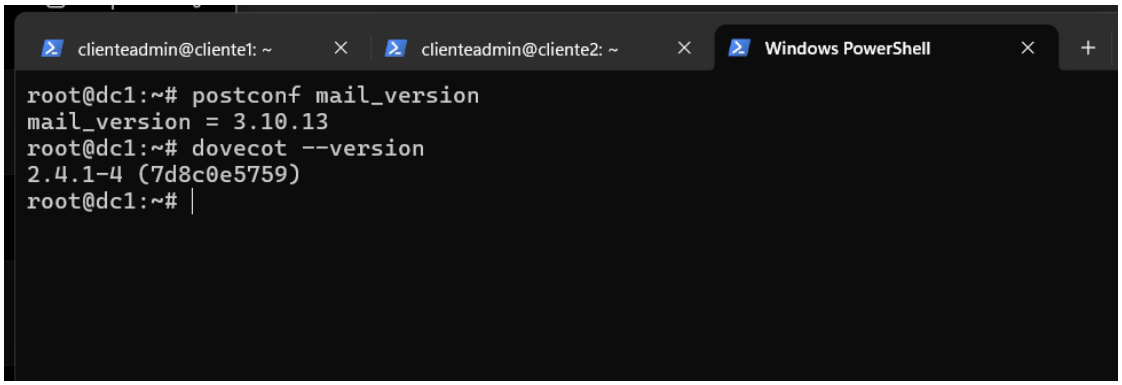
Implementación y validación del servicio de correo electrónico

Como último servicio principal del alcance se implementó un sistema de correo interno para el dominio redesi.test. Postfix funciona como MTA SMTP y Dovecot proporciona acceso IMAP y POP3. La entrega local utiliza buzones Maildir y los servicios de transporte y consulta se validaron con TLS utilizando el mismo certificado X.509 autofirmado de dc1.redesi.test creado previamente para el laboratorio.

Parámetro	Configuración final
Servidor de correo	dc1.redesi.test / 192.168.3.2
Dominio de correo	redesi.test
Postfix	3.10.13
Dovecot	2.4.1-4
SMTP	TCP/25 + STARTTLS
IMAP / IMAPS	TCP/143 / TCP/993
POP3 / POP3S	TCP/110 / TCP/995
Formato de buzón	Maildir: ~/Maildir
Certificado TLS	/etc/ssl/certs/vsftpd.crt – CN=dc1.redesi.test
Cuenta 1	correo1@redesi.test / CorreoRedes2026A
Cuenta 2	correo2@redesi.test / CorreoRedes2026B
Registro MX	redesi.test → prioridad 10 → dc1.redesi.test

Figura 22

Versiones de Postfix y Dovecot instaladas en dc1.



Nota. La salida confirma Postfix 3.10.13 y Dovecot 2.4.1-4 sobre Debian 13.6.

Cuentas de correo y estructura Maildir

Se crearon dos usuarios locales independientes para demostrar envío y recepción bidireccional. Cada cuenta posee un directorio ~/Maildir con los subdirectorios cur, new y tmp, propiedad de su usuario y con permisos restrictivos.

```
        useradd -m -s /bin/bash correo1
passwd correo1        # CorreoRedes2026A
useradd -m -s /bin/bash correo2
passwd correo2        # CorreoRedes2026B


mkdir -p /home/correo1/Maildir/{cur,new,tmp}
mkdir -p /home/correo2/Maildir/{cur,new,tmp}
chown -R correo1:correo1 /home/correo1/Maildir
chown -R correo2:correo2 /home/correo2/Maildir
chmod -R 700 /home/correo1/Maildir
chmod -R 700 /home/correo2/Maildir
```

Configuración de Postfix: SMTP, Maildir y STARTTLS

Postfix se configuró con redesi.test como nombre de correo del sistema y dc1.redesi.test como hostname. La entrega local se cambió a Maildir/ y STARTTLS se habilitó en SMTP con el

certificado de dc1. La política smtpd_tls_security_level = may anuncia STARTTLS en el puerto 25; las pruebas de cliente utilizaron explícitamente TLS.

```
        home_mailbox = Maildir/

myhostname = dc1.redesi.test

mydestination = $myhostname, redesi.test, dc1.redesi.test, localhost.redesi.test, localhost

smtpd_tls_cert_file = /etc/ssl/certs/vsftpd.crt

smtpd_tls_key_file = /etc/ssl/private/vsftpd.key

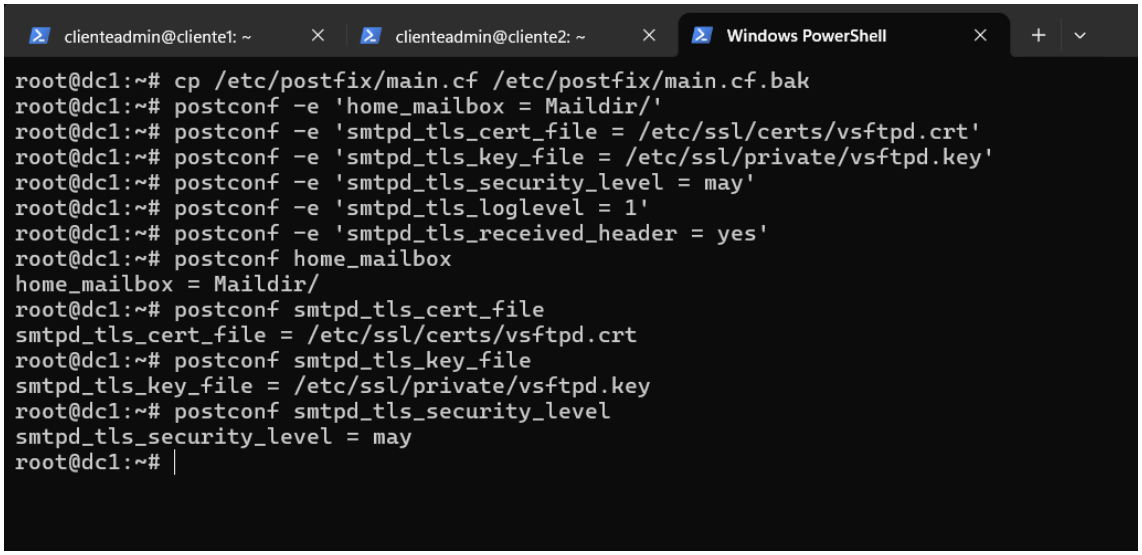
smtpd_tls_security_level = may

smtpd_tls_loglevel = 1

smtpd_tls_received_header = yes
```

Figura 23

Configuración de Postfix para Maildir y TLS.



Nota. Se observan home_mailbox=Maildir/, el certificado y la clave privada del laboratorio, y smtpd_tls_security_level=may.

Configuración de Dovecot: IMAP, POP3 y TLS

Dovecot se configuró para utilizar Maildir, autenticación PAM de usuarios locales y cifrado obligatorio. Se habilitaron IMAP y POP3, incluyendo IMAPS en 993 y POP3S en 995. Durante las pruebas se detectó que la directiva heredada mail_inbox_path = /var/mail/%{user} hacía que INBOX se interpretara de forma incorrecta. Se eliminó esa directiva de 10-mail.conf y se

mantuvieron únicamente mail_driver = maildir y mail_path = ~/Maildir. Después de reiniciar Dovecot, INBOX pasó a /home/correo2/Maildir y los dos mensajes existentes fueron indexados correctamente.

```
        mail_driver = maildir
mail_path = ~/Maildir


ssl = required
ssl_server_cert_file = /etc/ssl/certs/vsftpd.crt
ssl_server_key_file = /etc/ssl/private/vsftpd.key


auth_mechanisms = plain login
auth_allow_cleartext = no


protocols {
  imap = yes
  pop3 = yes
}
```

```
        grep -Rns "mail_inbox_path" /etc/dovecot/conf.d
sed -i '/^[[:space:]]*mail_inbox_path[[:space:]]*=/d' /etc/dovecot/conf.d/99-redesi.conf
sed -i 's|^[[:space:]]*mail_inbox_path[[:space:]]*=.*|# mail_inbox_path deshabilitado - se usa
Maildir|' /etc/dovecot/conf.d/10-mail.conf
systemctl restart dovecot
doveadm mailbox path -u correo2 -t dir INBOX
# /home/correo2/Maildir
doveadm mailbox status -u correo2 all INBOX
# INBOX messages=2 ... vsize=1284
```

Servicios activos y puertos de escucha

Después de aplicar la configuración, systemctl confirmó Postfix y Dovecot como active (running). La comprobación con ss -tlnp mostró SMTP en 25, POP3 en 110, IMAP en 143, IMAPS en 993 y POP3S en 995, además de los servicios previos del controlador de dominio y FTPS.

```
        Postfix: active (running) – versión 3.10.13
Dovecot: active (running) – versión 2.4.1-4


TCP/25   SMTP / STARTTLS
TCP/110  POP3
TCP/143  IMAP
TCP/993  IMAPS
TCP/995  POP3S
```

Registro MX en Samba DNS

Se agregó un registro MX para el dominio redesi.test con prioridad 10 y destino dc1.redesi.test. La consulta local en el DNS de Samba y la consulta desde Cliente1 devolvieron el mismo resultado, mientras que dc1.redesi.test resolvió a 192.168.3.2.

```
        samba-tool dns add localhost redesi.test @ MX "dc1.redesi.test 10" -U Administrator
samba-tool dns query localhost redesi.test @ MX -U Administrator
host -t MX redesi.test 127.0.0.1


Resultado:
redesi.test mail is handled by 10 dc1.redesi.test.


Cliente1:
host -t MX redesi.test
redesi.test mail is handled by 10 dc1.redesi.test.
host dc1.redesi.test
dc1.redesi.test has address 192.168.3.2
```

Figura 24

Registro MX del dominio redesi.test en Samba DNS.



Nota. La consulta samba-tool y host confirma el MX con prioridad 10 apuntando a dc1.redesi.test.

Pruebas de verificación del servicio de correo

Después de completar Postfix, Dovecot, Maildir, TLS y el registro MX se realizaron pruebas locales y remotas. Las pruebas siguientes abarcan entrega local, SMTP con STARTTLS, lectura IMAPS, recuperación POP3S, envío en ambos sentidos y revisión de los registros de auditoría.

Prueba de entrega local de Postfix

Antes de probar los clientes se ejecutó una entrega local hacia correo2@redesi.test. El mensaje apareció en /home/correo2/Maildir/new, se pudo leer directamente y la cola de Postfix quedó vacía, demostrando que la entrega local Maildir funcionaba antes de introducir IMAP/POP3.

```
        echo "Primera prueba del servidor de correo REDESI.TEST" | mail -s "Prueba Postfix"
correo2@redesi.test
ls -la /home/correo2/Maildir/new
cat /home/correo2/Maildir/new/*
postqueue -p

Resultado:
Delivered-To: correo2@redesi.test
Subject: Prueba Postfix
Primera prueba del servidor de correo REDESI.TEST
Mail queue is empty
```

Prueba SMTP con STARTTLS: Cliente1 → Cliente2

Desde DebianClient1 se instaló swaks y se envió un mensaje desde correo1@redesi.test hacia correo2@redesi.test. El servidor anunció STARTTLS, negoció TLS 1.3 con TLS_AES_256_GCM_SHA384, aceptó MAIL FROM, RCPT TO y DATA y respondió 250 2.0.0 Ok. El log de Postfix registró la conexión desde 192.168.1.2 y status=sent (delivered to maildir).

```
        swaks --server dc1.redesi.test --port 25 --tls \
  --from correo1@redesi.test --to correo2@redesi.test \
  --header "Subject: Prueba SMTP TLS desde Cliente1" \
  --body "Mensaje enviado desde Cliente1 mediante SMTP con STARTTLS."

Evidencia principal:
250-STARTTLS
220 2.0.0 Ready to start TLS
TLS started with cipher TLSv1.3:TLS_AES_256_GCM_SHA384:256
MAIL FROM:<correo1@redesi.test> → 250 2.1.0 Ok
RCPT TO:<correo2@redesi.test> → 250 2.1.5 Ok
250 2.0.0 Ok: queued as 4BE99401BA
Postfix: status=sent (delivered to maildir)
```

Prueba IMAPS: lectura en Cliente2

Una vez corregida la ruta de INBOX, Dovecot detectó dos mensajes en correo2. Desde DebianClient2, curl sobre imaps://dc1.redesi.test recuperó primero el mensaje local y después el enviado desde Cliente1. El encabezado recibido conserva la evidencia de que el mensaje entró al servidor mediante ESMTPS y TLS 1.3.

```
        doveadm mailbox status -u correo2 all INBOX
INBOX messages=2 recent=2 uidnext=3 unseen=2 vsize=1284


Cliente2:
curl -k --user correo2:CorreoRedes2026B "imaps://dc1.redesi.test/INBOX/;MAILINDEX=2"


Return-Path: <correo1@redesi.test>
Delivered-To: correo2@redesi.test
Received: from cliente1.redesi.test (unknown [192.168.1.2])
  (using TLSv1.3 with cipher TLS_AES_256_GCM_SHA384 ...)
From: correo1@redesi.test
To: correo2@redesi.test
Subject: Prueba SMTP TLS desde Cliente1


Mensaje enviado desde Cliente1 mediante SMTP con STARTTLS.
```

Prueba POP3S: autenticación y recuperación en Cliente2

POP3S se validó de dos formas. OpenSSL negoció directamente TLS en el puerto 995, Dovecot respondió +OK, la cuenta correo2 autenticó correctamente y STAT/LIST mostraron dos mensajes. RETR 2 recuperó el correo de Cliente1. La misma recuperación se repitió con curl sobre pop3s://, confirmando el funcionamiento desde un segundo cliente.

```
        openssl s_client -connect dc1.redesi.test:995 -servername dc1.redesi.test -quiet
+OK Dovecot ready.
USER correo2
+OK
```

```
PASS CorreoRedes2026B

+OK Logged in.

STAT

+OK 2 1284

LIST

+OK 2 messages:

1 493

2 791

RETR 2

+OK 791 octets

From: correo1@redesi.test

To: correo2@redesi.test

Subject: Prueba SMTP TLS desde Cliente1

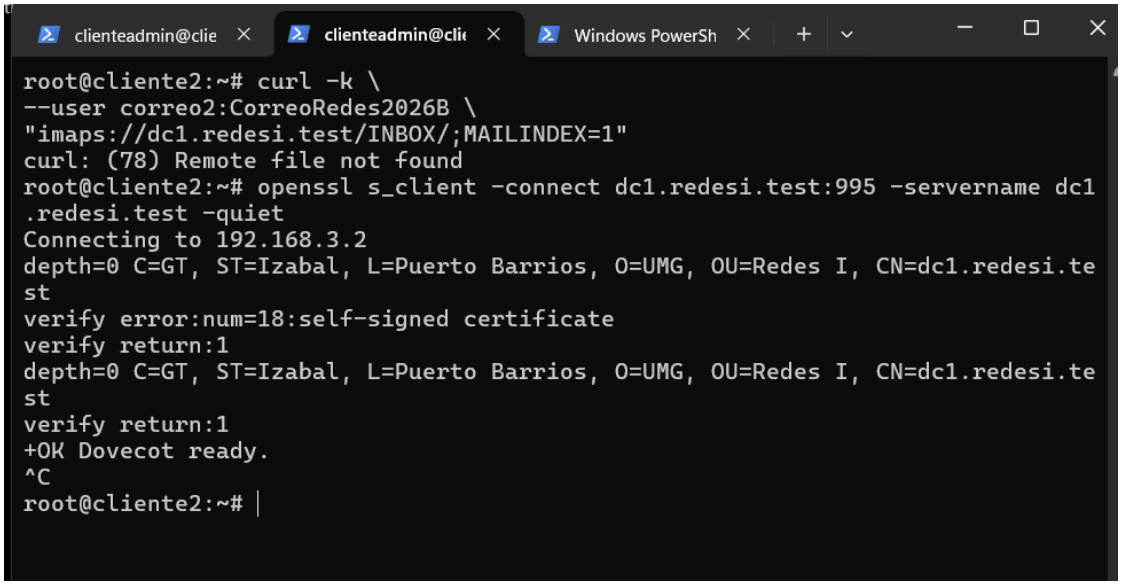
QUIT

+OK Logging out.


curl -k --user correo2:CorreoRedes2026B pop3s://dc1.redesi.test/2
```

Figura 25

Negociación TLS con POP3S en DebianClient2.



Nota. OpenSSL alcanza 192.168.3.2:995, valida el certificado autofirmado del laboratorio y recibe +OK Dovecot ready.

Prueba bidireccional: Cliente2 → Cliente1

Para demostrar que el servidor no funciona únicamente en una dirección, DebianClient2 envió un segundo mensaje mediante SMTP/STARTTLS desde correo2@redesi.test hacia correo1@redesi.test. Postfix lo aceptó como 1FFDC4016E, lo entregó a Maildir y DebianClient1 lo recuperó mediante IMAPS. Esta prueba confirma envío y recepción para ambas cuentas.

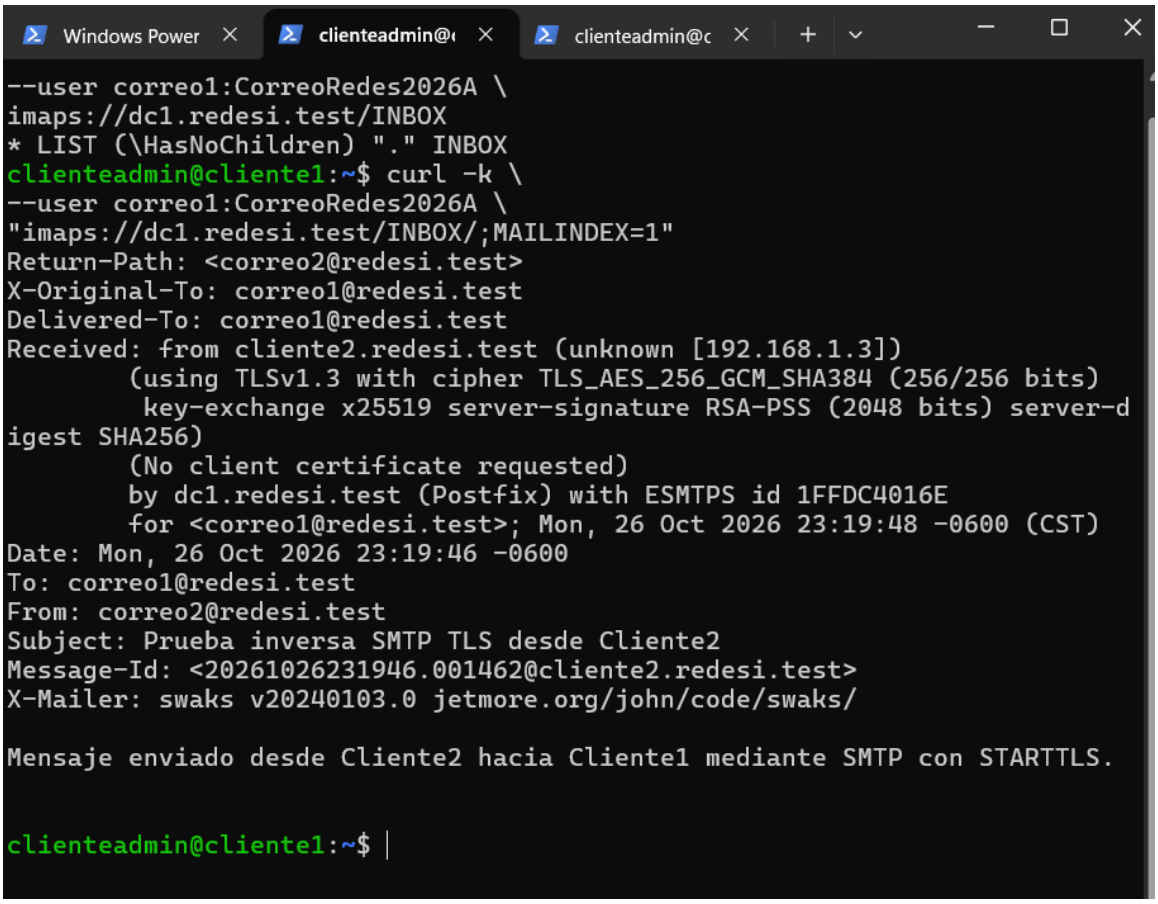
```
        Cliente2 – swaks:
TLSv1.3:TLS_AES_256_GCM_SHA384
MAIL FROM:<correo2@redesi.test> → 250 2.1.0 Ok
RCPT TO:<correo1@redesi.test> → 250 2.1.5 Ok
250 2.0.0 Ok: queued as 1FFDC4016E


Postfix en dc1:
from=<correo2@redesi.test>, nrcpt=1
to=<correo1@redesi.test>, relay=local, dsn=2.0.0, status=sent (delivered to maildir)


Cliente1 – IMAPS:
From: correo2@redesi.test
To: correo1@redesi.test
Subject: Prueba inversa SMTP TLS desde Cliente2
Mensaje enviado desde Cliente2 hacia Cliente1 mediante SMTP con STARTTLS.
```

Figura 26

Mensaje inverso recuperado por IMAPS en DebianClient1.



Nota. La evidencia muestra From correo2@redesi.test, To correo1@redesi.test, TLS 1.3 en el encabezado Received y el cuerpo enviado desde Cliente2.

Registros finales de Postfix y Dovecot

Los registros del servidor corroboran las pruebas de cliente. Postfix registra conexiones TLS 1.3 desde 192.168.1.2 y 192.168.1.3, junto con status=sent (delivered to maildir) para ambos sentidos. Dovecot registra inicios de sesión IMAP y POP3 con TLS, recuperación de cuerpos y cierre normal de las sesiones. Los estados finales de los buzones muestran un mensaje en correo1 y dos mensajes en correo2.

```
        Postfix:
192.168.1.2 → Anonymous TLS connection established ... TLSv1.3 / TLS_AES_256_GCM_SHA384
correo1@redesi.test → correo2@redesi.test → status=sent (delivered to maildir)
192.168.1.3 → Anonymous TLS connection established ... TLSv1.3 / TLS_AES_256_GCM_SHA384
correo2@redesi.test → correo1@redesi.test → status=sent (delivered to maildir)

Dovecot:
imap-login: user=<correo2> ... rip=192.168.1.3 ... TLS
pop3-login: user=<correo2> ... rip=192.168.1.3 ... TLS
imap-login: user=<correo1> ... rip=192.168.1.2 ... TLS

Buzones finales:
correo1: INBOX messages=1, vsize=814
correo2: INBOX messages=2, vsize=1284
```

Matriz consolidada de pruebas del servicio de correo

Elemento comprobado	Prueba / comando	Resultado observado	Conclusión
Postfix activo	systemctl status postfix	active (running), versión 3.10.13	MTA operativo
Dovecot activo	systemctl status dovecot	active (running), versión 2.4.1-4	IMAP/POP3 operativo
Puertos	ss -tlnp	25, 110, 143, 993 y 995 LISTEN	Servicios publicados
MX DNS	host -t MX redesi.test	10 dc1.redesi.test	Enrutamiento de correo DNS correcto
Entrega Maildir	mail + ls Maildir/new	Mensaje creado; queue empty	Entrega local correcta
SMTP TLS C1→C2	swaks --tls	TLS 1.3; 250 queued; delivered to maildir	Envío seguro correcto
IMAPS C2	curl imaps://... MAILINDEX=2	Mensaje correo1→correo2 recuperado	Lectura IMAPS correcta

Elemento comprobado	Prueba / comando	Resultado observado	Conclusión
POP3S C2	OpenSSL / curl pop3s	STAT 2; RETR 2; mensaje recuperado	POP3S correcto
SMTP TLS C2→C1	swaks --tls	TLS 1.3; queued 1FFDC4016E	Envío inverso correcto
IMAPS C1	curl imaps://...	Mensaje correo2→correo1 recuperado	Recepción inversa correcta
Auditoría Postfix	journalctl -u postfix	status=sent (delivered to maildir)	Entrega registrada
Auditoría Dovecot	journalctl -u dovecot	imap-login/pop3-login con TLS	Acceso registrado

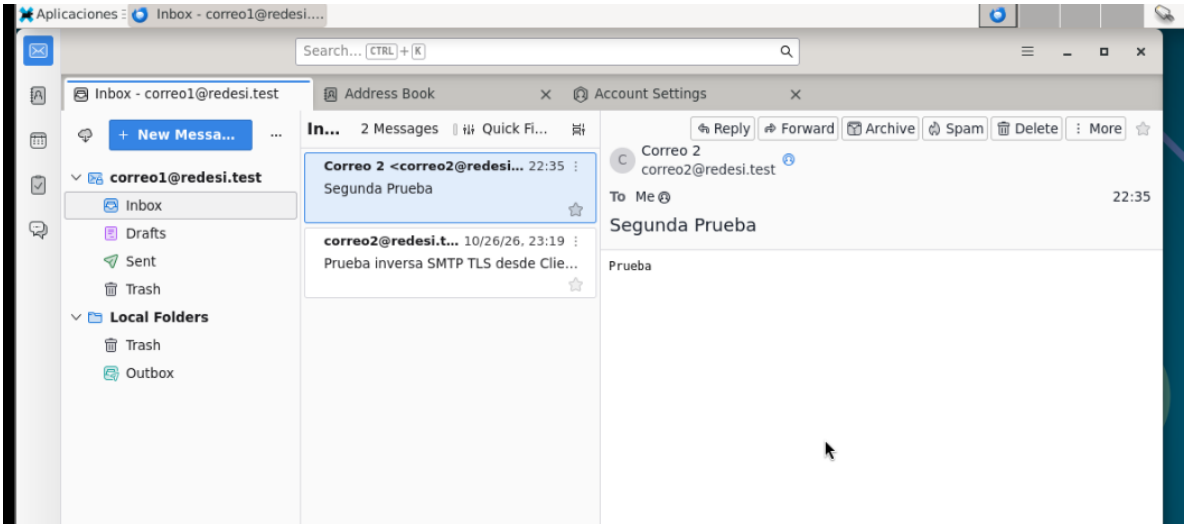
Demostración visual del correo con Thunderbird

Para la demostración final se instaló Thunderbird en DebianClient1 y DebianClient2. La recepción se configuró mediante IMAPS y el envío mediante SMTP con STARTTLS. Debido a que el certificado de dc1 es autofirmado, Thunderbird almacena una excepción de seguridad únicamente para dc1.redesi.test dentro del laboratorio.

Parámetro	Configuración / resultado
Cuenta Cliente1	correo1@redesi.test / CorreoRedes2026A
Cuenta Cliente2	correo2@redesi.test / CorreoRedes2026B
Entrada IMAP	dc1.redesi.test · TCP/993 · SSL/TLS · contraseña normal
Salida SMTP	dc1.redesi.test · TCP/25 · STARTTLS · sin autenticación SMTP
Carpetas	Inbox, Sent, Drafts y Trash asociadas a cada cuenta cuando corresponde

Figura 29

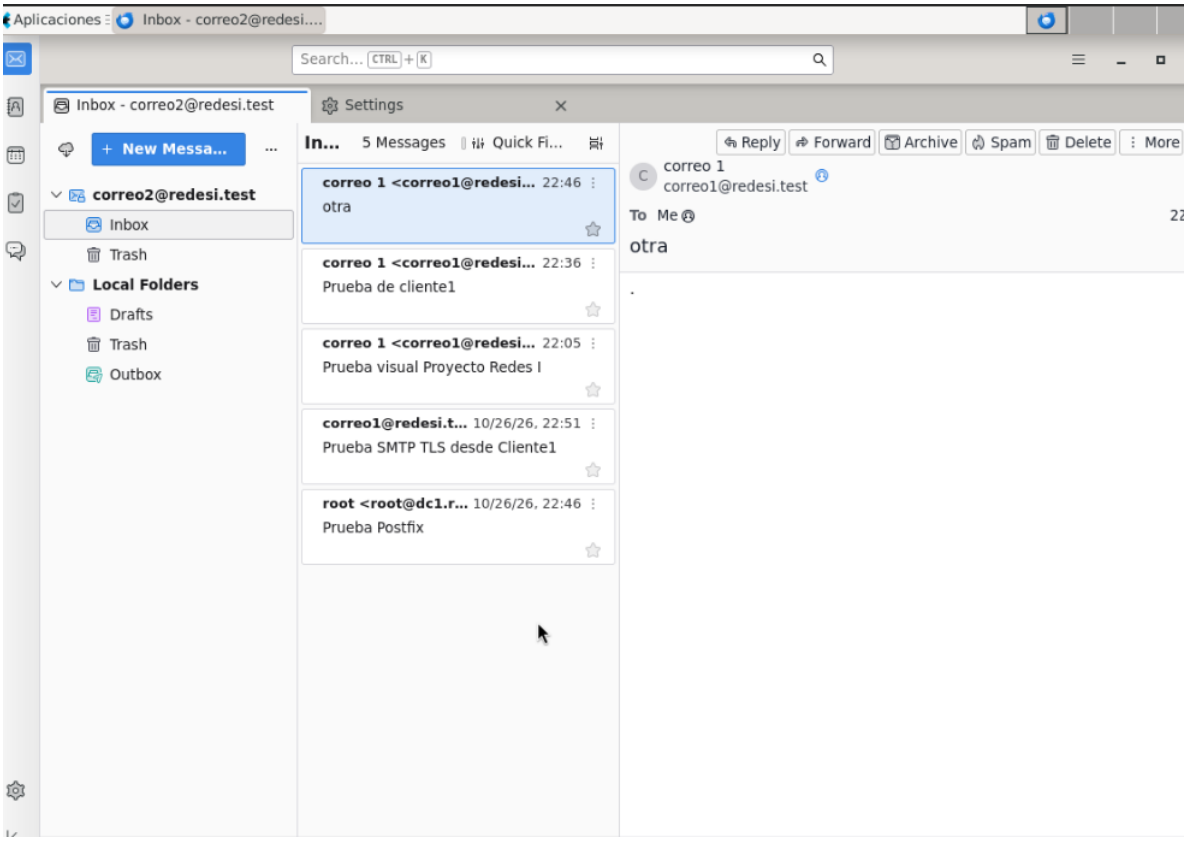
Recepción en Thunderbird de correo2 → correo1 en DebianClient1.



Nota. La bandeja de correo1 muestra los mensajes recibidos desde correo2, incluida la prueba inversa SMTP/TLS realizada desde Cliente2.

Figura 30

Recepción en Thunderbird de correo1 → correo2 en DebianClient2.



Nota. La bandeja de correo2 muestra los mensajes enviados desde correo1 y confirma visualmente el funcionamiento extremo a extremo del servidor de correo.

Resultado final del servicio de correo

Resultado final: el servicio de correo electrónico se considera COMPLETADO. El dominio redesi.test dispone de registro MX hacia dc1.redesi.test; Postfix entrega correo local en Maildir y negocia SMTP con STARTTLS; Dovecot proporciona IMAP/IMAPS y POP3/POP3S con TLS; y las cuentas correo1 y correo2 fueron probadas en ambos sentidos entre DebianClient1 y DebianClient2.

Estado consolidado del proyecto

Componente / requerimiento	Estado	Observación
Topología GNS3 y direccionamiento	Completado	Tres redes IPv4 y equipos actuales documentados.
Enrutamiento R1/R2	Completado	Conectividad entre LAN de clientes y LAN de servidores.
Salida a Internet por NAT1	Completado	R1 centraliza PAT para las redes internas.
Controlador de dominio Samba AD	Completado	dc1.redesi.test operativo.
DNS corporativo	Completado	Resolución interna y reenvío externo funcional.
Kerberos	Completado	Tickets válidos para usuarios del dominio.
cliente1 unido al dominio	Completado	SSSD/PAM/LightDM validados.
cliente2 unido al dominio	Completado	Cuenta de equipo independiente y autenticación validada.
usuario1	Completado	Cuenta de prueba del dominio.
usuario2	Completado	Creado y probado en cliente2.
DHCP IPv4	Completado	ISC DHCP Server activo en dc1, relay en R1 y reservas .2/.3 validadas con DHCPACK.
FTP seguro con TLS	Completado	vsftpd operativo en dc1; TLS 1.3, autenticación, modo pasivo, subida/descarga y bloqueo anónimo validados.
Correo SMTP + IMAP/POP3	Completado	Postfix + Dovecot operativos; MX, SMTP/STARTTLS, IMAPS y POP3S validados en ambos sentidos.

Componente / requerimiento	Estado	Observación
DNS MX de redesi.test	Completado	Prioridad 10 hacia dc1.redesi.test; validado desde servidor y cliente.
SMTP + STARTTLS	Completado	TLS 1.3 y entrega Maildir validados desde cliente1 y cliente2.
IMAP / IMAPS	Completado	Dovecot 993/TLS; lectura de buzones comprobada en ambos clientes.
POP3 / POP3S	Completado	Dovecot 995/TLS; STAT, LIST y RETR comprobados.

Actualización final del estado: la fila de correo se actualiza a Completado después de validar MX, Postfix, SMTP con STARTTLS, Dovecot, IMAPS, POP3S y envío bidireccional. Las menciones anteriores a servicios “pendientes” se conservan como registro cronológico del avance del proyecto.

Próximos pasos

La entrega semanal de DHCP quedó completada. Se validaron las reservas de cliente1 y cliente2, el relay en R1, la entrega de gateway, DNS y dominio, los registros DHCPACK y la conectividad con dc1, Internet y Active Directory. Las evidencias y configuraciones finales se incorporaron en esta actualización del documento.

Después de completar DHCP se implementaron y validaron FTPS y el sistema de correo. En el estado actual todos los servicios principales del alcance están completados. Los pasos restantes corresponden únicamente al cierre de la entrega: exportar la versión actualizada del proyecto GNS3, organizar las capturas de evidencia, actualizar el repositorio y preparar la demostración integral.

Actualización final. FTPS y correo electrónico ya fueron completados y probados desde ambos clientes. La documentación incorpora pruebas técnicas por servicio y evidencias visuales con FileZilla y Thunderbird para facilitar la exposición.

Cierre técnico y tareas restantes de entrega

Actualización final. Con la validación del sistema de correo, todos los servicios principales definidos para el proyecto se encuentran completados: enrutamiento y NAT, controlador de dominio Samba AD, DNS, Kerberos, integración de clientes, DHCP con relay, FTPS con TLS y correo mediante SMTP/STARTTLS, IMAPS y POP3S. Las tareas restantes ya no corresponden a configuración de servicios, sino al cierre de la entrega: exportar la versión actual de GNS3, sustituir el archivo grande del repositorio por el enlace de Google Drive, subir esta documentación final y realizar una demostración integral con los equipos encendidos.

La demostración gráfica con FileZilla y Thunderbird se agregó como evidencia complementaria. Estas pruebas no reemplazan las verificaciones de protocolo registradas con curl, OpenSSL, swaks, journalctl y los comandos de Samba/DHCP; las complementan con una validación visual orientada a la exposición del proyecto.

Apéndice A. Comandos de verificación del servicio de correo

```
         dc1:
postconf -n
doveconf -n
systemctl status postfix --no-pager
systemctl status dovecot --no-pager
ss -tlnp
host -t MX redesi.test 127.0.0.1
doveadm mailbox status -u correo1 all INBOX
doveadm mailbox status -u correo2 all INBOX
journalctl -u postfix -n 50 --no-pager
journalctl -u dovecot -n 50 --no-pager


cliente1 / cliente2 – DNS:
host -t MX redesi.test
```

```
host dc1.redesi.test


SMTP STARTTLS:

swaks --server dc1.redesi.test --port 25 --tls --from correo1@redesi.test --to correo2@redesi.test --header "Subject:

Prueba SMTP TLS" --body "Prueba"


IMAPS:

curl -k --user correo2:CorreoRedes2026B "imaps://dc1.redesi.test/INBOX/;MAILINDEX=2"


POP3S:

openssl s_client -connect dc1.redesi.test:995 -servername dc1.redesi.test -quiet

USER correo2

PASS CorreoRedes2026B

STAT

LIST

RETR 2

QUIT
```

Apéndice B. Conexiones SSH del laboratorio

```
         GNS3 VM:

ssh gns3@192.168.56.102


dc1:

ssh -J gns3@192.168.56.102 -p 2223 root@192.168.122.100


cliente1:

ssh -J gns3@192.168.56.102 -p 2222 clienteadmin@192.168.122.100


cliente2:

ssh -J gns3@192.168.56.102 -p 2224 clienteadmin@192.168.122.100


Elevar a root en clientes:

su -


Credenciales de salto: gns3 / gns3

Cliente local: clienteadmin / ClienteAdmin2026A

Root clientes: root / DebianRoot2026A
```

Apéndice C. Comandos de verificación utilizados

```
Servidor dc1:
 hostname -f
 systemctl status samba-ad-dc --no-pager
 samba-tool user list
 samba-tool user show usuario2
 host dc1.redesi.test
 host -t SRV _ldap._tcp.redesi.test
 host -t SRV _kerberos._udp.redesi.test

Clientes:
 realm list
 id usuario1
 id usuario2
 whoami
 hostname -f
 kinit usuario1   # o usuario2
 klist

R1:
 show ip route 192.168.3.2
 show ip nat translations
 telnet 192.168.3.2 22
```

Comandos adicionales de DHCP y administración remota:

```
        dc1:
ip route get 192.168.122.1
dhcpd -t -cf /etc/dhcp/dhcpd.conf
systemctl status isc-dhcp-server --no-pager
journalctl -u isc-dhcp-server -n 100 --no-pager

R1:
show running-config interface FastEthernet0/0

Clientes:
nmcli dev show enp2s0
ip -4 addr show enp2s0
```

Apéndice D. Comandos de verificación de FTPS

```
          dc1:
 systemctl status vsftpd --no-pager
 ss -tlnp
 openssl x509 -in /etc/ssl/certs/vsftpd.crt -noout -subject -dates
 ls -la /home/ftpuser/ftp/files
 tail -n 50 /var/log/vsftpd.log

 cliente1 / cliente2:
 openssl s_client -connect dc1.redesi.test:21 -starttls ftp -servername dc1.redesi.test
 curl --ssl-reqd --insecure -u ftpuser:FTPRedes2026A ftp://dc1.redesi.test/files/
 curl --ssl-reqd --insecure -u ftpuser:FTPRedes2026A -T cliente1_ftps.txt ftp://dc1.redesi.test/files/
 curl --ssl-reqd --insecure ftp://dc1.redesi.test/files/
```